IUT Nantes | GCE Electronics

RAPPORT DE STAGE

Sujet :

Conception d'une extension pour un automate connecté

Noah PERRET

2ème année BUT GEII

2025

Conception de l'extension EBX-LINK pour un automate connecté

Du 6 janvier au 14 mars 2025

Rapport réalisé dans le cadre de mon stage sous la supervision de Monsieur Jérémy Bergeron, Responsable recherche et développement.

Bon pour la diffusion, le 19/03/2025



Tuteur de stage : Jérémy BERGERON

Tuteur pédagogique : MOUCHÈRE Harold

Etablissement : IUT de Nantes campus de Carquefou

Formations : Génie Électrique et Informatique Industrielle

Entreprise d'accueil : GCE Electronics - 9 Rue Jean Mermoz, 44980 SAINTE-LUCE-SUR-LOIRE

Page de remerciements

Je tiens tout d'abord à remercier Jérémy Bergeron mon maître de stage, qui m'a fait visiter l'entreprise en premier lieu, mais aussi pour son travail concernant mes questions relatives au cahier des charges et au fonctionnement logiciel des appareils de GCE Electronics. Je le remercie également pour ses relectures du code que j'ai produit durant le stage.

Un grand merci aussi à Harold Mouchère pour son aide précieuse concernant mes questions tout au long du stage, ainsi que pour son soutien sur ce rapport et la préparation de la soutenance. Merci également à l'équipe pédagogique, notamment Sébastien Levilly, pour son aide dans la recherche de ce stage.

Bien entendu, je remercie également Patrick Gorce, le dirigeant de GCE Electronics, qui m'a chaleureusement accueilli dans son entreprise. Il m'a beaucoup aidé à améliorer la CEM de mes cartes électroniques en m'enseignant les grandes règles qui me seront utiles tout au long de ma carrière. Un grand merci aussi à Christèle Gorce pour sa réactivité face à mes demandes par mail ainsi que pour sa gentillesse.

Je tiens à remercier Kévin, véritable couteau suisse de l'entreprise, qui a répondu à toutes mes questions et demandes, que ce soit durant la fabrication ou la conception de ma carte électronique.

J'ai également reçu de l'aide des deux experts de l'atelier, Axel et Tom, qui m'ont expliqué leurs différents rôles, et notamment Axel pour son aide durant sérigraphie.

Merci aussi à Maxence pour son aide précieuse sur la partie Web OTA, ainsi que pour les souvenirs de ses combats fratricides contre Tom aux jeux de société.

Enfin, je souhaite également remercier Bruno, qui a été mon colocataire de bureau durant toute la durée de mon stage.

Glossaire

Ce glossaire regroupe les termes techniques et spécifiques à GCE Electronics utilisés dans ce document afin de faciliter leur compréhension.

- **PCB (Printed Circuit Board) :** Carte de circuit imprimé permettant l'interconnexion des composants électroniques.
- **EBX Connect & Power :** Système développé par GCE Electronics permettant une connexion et une alimentation optimisées des équipements RJ45.
- **EBX Mesh :** Réseau maillé basé sur le protocole EBX, utilisé pour la communication sans fil entre les appareils connectés de chez GCE.
- **BLE (Bluetooth Low Energy) :** Version basse consommation du Bluetooth, utilisée pour les communications sans fil à courte portée.
- **RoHS (Restriction of Hazardous Substances) :** Directive européenne limitant l'utilisation de substances dangereuses dans les équipements électroniques.
- **ESD (ElectroStatic Discharge) :** Décharge électrostatique pouvant endommager les composants électroniques sensibles.
- **Proteus ARES (Advanced Routing and Editing Software) :** Logiciel de conception de circuits imprimés permettant le routage et la simulation de circuit électronique.
- **Stencil :** Pochoir utilisé pour l'application de la pâte à braser sur les PCB avant le montage des composants CMS.
- **Callback :** Fonction de rappel utilisée en programmation, permettant d'exécuter un morceau de code en réponse à un événement.
- **UART (Universal Asynchronous Receiver-Transmitter) :** Protocole de communication série asynchrone permettant l'échange de données périphériques
- **OTA (Over The Air) :** Méthode de mise à jour ou de transmission de données sans fil, souvent utilisée pour les mises à jour de firmware ou de logiciels sur des appareils connectés.

Dans ce rapport de stage, vous pourrez retrouver les symboles ci-dessous représentant les 3 principaux types de compétence :



🔍 Vérifier : Regroupe tout ce qui concerne les tests, la validation et l'analyse des systèmes.

🛠️ Concevoir : Englobe tout ce qui touche au développement, à la schématisation et à la conception technique.

🔁 Maintenir : Pour tout ce qui concerne la maintenance, l'optimisation et l'amélioration des systèmes après leur développement initial.

Ces trois axes structureront le rapport et permettront une meilleure compréhension ces missions réalisées au sein de l'entreprise, tout en reliant les compétences apprises à l'IUT à celles développées et utilisées durant le stage.

Sommaire

3	Page de remerciement
4	Glossaire
6	Introduction
7	1 – Préambule : 1.1 GCE Electronics l'innovation made in France 1.2 Une interface Bluetooth, Mesh et RJ45! 1.3 Cahier des charges et spécifications fonctionnelles
11	2 – Conception électronique
21	3 - La proof of concept
26	4 - Le début du développement
28	5 - Réception du PCB et réalisation de prototype
32	6 - La suite du développement logiciel
37	7 - Le développement WEB
39	8 - Les tests et l'optimisation logicielle
42	Conclusion : Bilan du travail réalisé durant ces 10 semaines Bilan personnel et perspectives professionnelle
43	Annexes
47	Bibliographie
48	Résumé

Introduction

Avec la montée en puissance de l'électronique embarquée, des objets connectés et des solutions domotiques intelligentes ces quinze dernières années. GCE Electronics, acteur français, a su passer de petite entreprise nouvelle dans le secteur, a reconnu dans le domaine de l'électronique connectée. Elle s'est démarquée face à la concurrence asiatique féroce, en innovant constamment pour répondre au besoin toujours plus grand de ses acheteurs.

C'est dans ce contexte que mon stage s'inscrit, avec pour mission la conception d'une nouvelle extension, l'EBX-LINK. Elle est destinée à améliorer l'intercommunication des équipements de la marque en intégrant des technologies de communication sans-fil, comme le réseau Mesh et le Bluetooth. Ce qui facilite ainsi l'installation et la configuration du matériel de GCE Electronics pour le grand public.

C'est pourquoi mon stage s'est articulé autour de la problématique suivante :
Comment concevoir une carte électronique permettant une communication efficace entre des réseaux multisupport : Bluetooth Mesh et RJ45 ?

Pour répondre à cette question, mon travail s'est structuré en plusieurs étapes :

Tout d'abord, nous étudieront les besoins du projet et nous définirons le cahier des charges précis, en tenant compte des spécifications fonctionnelles et des contraintes d'intégration avec les équipements existants.

Ensuite, nous étudierons la conception matérielle à travers le développement du PCB, la gestion de l'alimentation et l'optimisation du routage pour garantir la compatibilité électromagnétique.

Une fois ces étapes complétées, la fabrication et l'assemblage des premiers prototypes ont été réalisés. Cela comprenait l'impression des circuits imprimés, l'assemblage des composants et les premiers tests en atelier pour s'assurer du bon fonctionnement avant l'intégration logicielle.

Le développement logiciel a ensuite porté sur la programmation des ESP32, en mettant en place des protocoles de communication tels que Bluetooth Mesh et UART via RS485.

Enfin, une phase de tests et d'optimisation a permis de vérifier les performances de la carte en conditions réelles, mais aussi en condition extrême. Dans le but d'améliorer la robustesse ainsi que la stabilité de l'EBX-Link.

Ce rapport détaillera ainsi chaque étape du projet, depuis la conception jusqu'aux tests finaux, en mettant en avant les défis rencontrés et les solutions mises en place.

1 - Préambule

1.1 GCE Electronics l'innovation made in France

Fondée en 2009 par Patrick, GCE Electronics est une PME spécialisée dans l'électronique connectée. Pionnière dans son domaine, elle est reconnue pour son expertise et son savoir-faire dans la conception et la fabrication de solutions domotiques. Basée à Sainte-Luce-sur-Loire, l'entreprise a su conserver un esprit familial et un cadre de travail agréable, tout en ayant un résultat net de plus de 180 000 € en 2023.

GCE Electronics est aussi engagée dans la réindustrialisation de la France, en effet l'entreprise tient à cœur de garder un maximum de ses outils de production pour ses cartes électroniques et d'usinage en France. Pour cela, elle a sa propre ligne de production avec ses nombreuses machines, qui sont régulièrement mises à jour afin de rester compétitives et d'être compatibles avec les dernières technologies et procédés.







L'entreprise se démarque également dans le suivi de la consommation énergétique et les capteurs connectés grâce à son automate phare l'IPX 800. Un automate connecté qui permet de relier plus d'une vingtaine d'extensions pour contrôler facilement toute une maison depuis une simple interface web. Le tout en développant des technologies innovantes comme son réseau EBX (Sans fil et Filaire). (Retrouver des détails sur les extensions annexe 5)



En outre, GCE Electronics respecte des normes environnementales et de sécurité stricte, comme la RoHS qui limite l'utilisation de substances dangereuses pour l'homme et l'environnement, la norme Européenne CE, mais aussi la norme 94-V0 pour la résistance au feu, ainsi que la certification UR des laboratoires « Underwriters » (UL) qui atteste de la sécurité des composants. De plus, l'entreprise met en œuvre un protocole ESD pour protéger ses produits contre des décharges électrostatiques durant leur production, garantissant ainsi leur durabilité et leur fiabilité.



Enfin, l'entreprise entreprend une politique éthique envers ses collaborateurs, en développant des outils sans cloud, sans abonnement et avec des mises à jour gratuites.

1.2 Une interface Bluetooth, Mesh et RJ45 !

L'entreprise GCE propose déjà un large panel de produits et d'extensions : d'une extension pour contrôler sa piscine (X-Pool) à une extension pour ouvrir ses volets (X-4VR) !



Tous ces produits fonctionnent avec une pluralité de réseaux et de protocoles : BLE Radio RJ45. Ce qui complique leur intégration dans une maison en limitant leur intercommunication. L'objectif de l'extension EBX-Link est de créer une passerelle solide entre ses différents supports, sans rendre plus difficile la prise en main du produit et son côté intuitif.

Afin de répondre à la problématique posée dans l'introduction, j'ai été amené, durant mon stage, à créer une carte électronique et à développer un logiciel permettant la communication sans fil via les réseaux EBX Mesh et Bluetooth. La carte permettra de faire passerelle entre les réseaux sans fil et les appareils filaires déjà vendus de chez GCE Electronics.

Voici le schéma des différentes interactions entre les deux principaux microcontrôleurs de la carte et les réseaux BLE et EBX Mesh. On peut voir, que celle-ci permet de mettre en relation une multitude de réseaux et d'appareils :

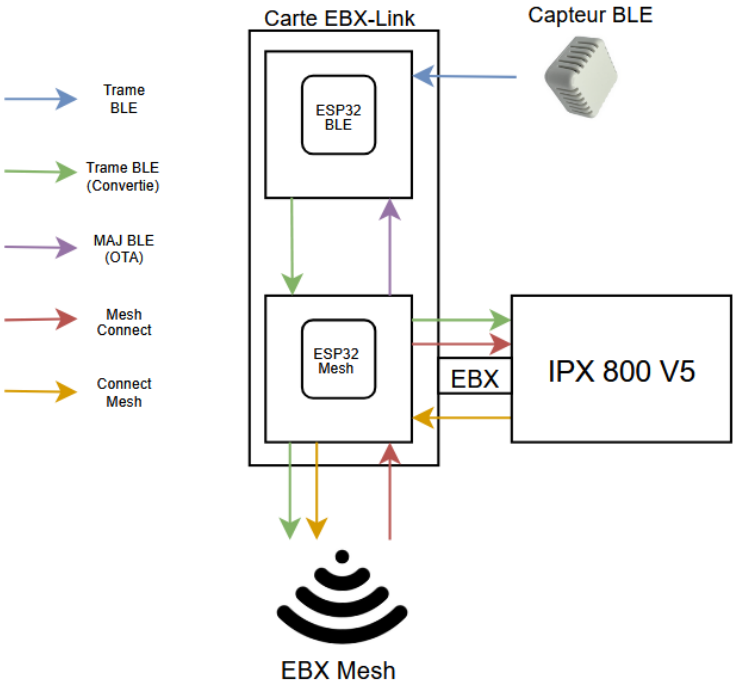


Figure 1: Schéma des interactions de l'EBX-Mesh dans un environnement complet

1.3 Cahier des charges et spécifications fonctionnelles

Pour mener à bien le projet, le cahier des charges a été fourni par Jérémy, l'ensemble des recommandations de celui-ci ont dû être respectées.

Le projet s'articule en deux axes de développement majeurs :

- Développement matériel :
  - Conception d'un PCB (4 couches) et de son prototype intégrable dans un boîtier au format rail DIN (figure 2), basé sur le hardware de projets précédents de l'entreprise.
  - Afin de développer les 2 réseaux sans fil, 2 ESP32 différents sont nécessaires.
  - Alimenter et connecter avec 2 ports EBX Powered + Connect.
  - Bouton et LED pour interagir avec l'ESP Mesh.

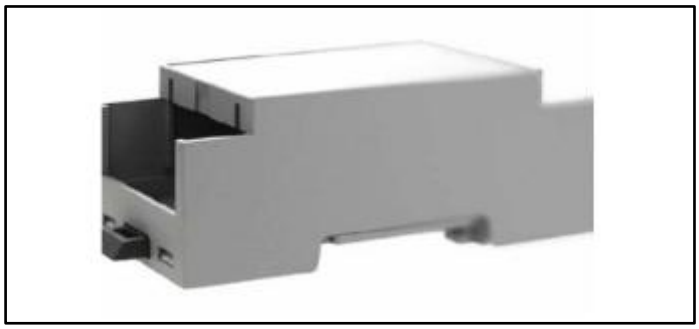


Figure 2 : Boîtier au format rail DIN pour rail TS-35 norme DIN 46277/3

- Développement logiciel :
  - Connectivité EBX Mesh radio pour les extensions compatibles (écrans connectés).
  - Intégration avec les capteurs Bluetooth (luminosité, humidité, température …)
  - Conception d'une passerelle connectée pour le contrôle à distance des équipements
  - 2 modes de fonctionnement pour l'ESP BLE : OTA pour les mises à jour et BLE pour la réception des capteurs de température
  - 3 modes pour le Mesh : Sleep lorsque le wifi est désactivé, Mesh pour le réseau radio EBX mesh et OTA pour les mises à jour

2 – Conception électronique

Bien que cela ne soit pas le plus commun, la première étape dans le développement de l'extension EBX-Link fut la conception du PCB. Cette étape, essentielle permettra de définir la structure électronique et les composants nécessaires permettant l'intégration des différentes fonctionnalités voulues.

2.1 Du RepeatDAC et BEACON au PCB de l'EBX-Link

Concevoir

Pour commencer cette conception de l'extension EBX-Link. J'ai dû me baser sur le hardware du repeatDAC (voir figure X). Cette carte intègre principalement deux ESP32, qui permettent à celle-ci, à la fois de servir de répéteur sur le réseau EBX-Mesh, mais aussi de convertir les trames Bluetooth reçues des capteurs vers le réseau Mesh.

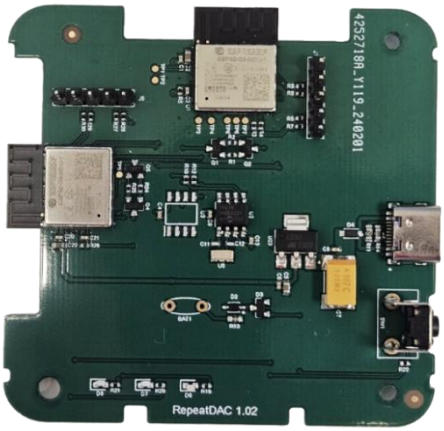


Figure 3 : Carte repeatDAC

Pour ma carte finale, j'ai repris les deux ESP32 ainsi que le bouton afin de respecter le cahier des charges. En outre, l'une des consignes principales était que mon PCB devait s'adapter à un boîtier au format rail DIN à l'instar d'un produit de chez GCE. En plus de cela, sur ce produit j'ai repris les ports RJ45 EBX–Power et Connect ainsi que le module de communication RS485.



Figure 4 : Boîtier au format rail DIN d'un ancien produit de chez GCE

Afin de respecter cette contrainte, une ébauche de PCB m'a été fournie sur Proteus ARES (), conçue avec les dimensions adaptées au boîtier, ce qui m'a permis de partir directement de cette base. J'ai ensuite schématisé sur draw.io le placement des différents modules et composants sur la carte (annexe 3), ce qui permettra de faciliter les prochaines étapes de routages.

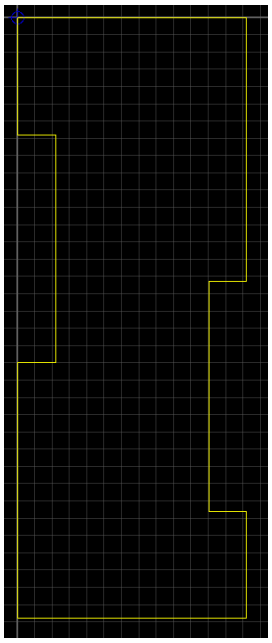


Figure 5 : Contour du PCB réalisé par Kevin

2.2 L'alimentation en énergie de l'extension

L'alimentation de la carte s'effectuera via l'un des 2 ports RJ45 compatibles EBX Connect de la carte. Ces ports seront situés aux extrémités inférieure et supérieure de la carte.



Figure 6 : Port RJ45 compatible EBX Powered et Connect

En entrée, de ces ports il y aura du 15 V, sachant que nos ESP32 ainsi que tous les composants principaux de la carte nécessitent une tension d'alimentation de 3.3V, il est indispensable d'abaisser cette tension. Pour cela, Patrick m'a demandé de m'inspirer d'un module d'alimentation d'une autre extension, le X-dimmer (figure 6).



Figure 7 : Module de contrôle d'éclairage (X-dimmer)

Le schéma électronique de l'alimentation a été intégré à celui de l'EBX-Link.

Cependant, ce module transformait une tension de 15 V en 5 V, les résistances du montage ont dû être adaptées afin d'avoir une tension de sortie de 3.3 V. Pour cette tension, a été déterminé qu'une résistance de 31,6 kΩ était requise pour R18.

Au niveau de l'implémentation, j'ai suivi exactement la même disposition que sur la carte de référence afin que l'alimentation soit optimisée en termes d'espace et de protections CEM.

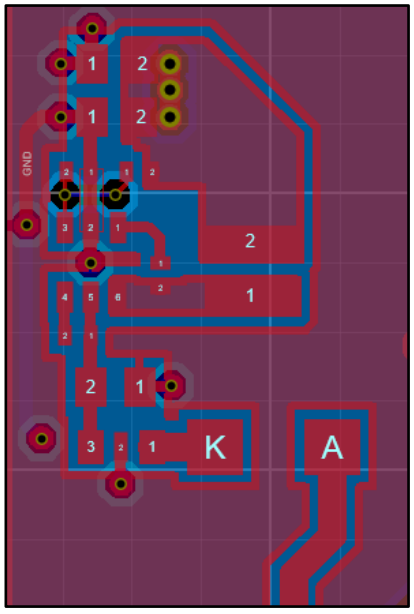


Figure 8 : Alimentation située, à l'extrémité inférieure gauche du PCB

2.3 La Communication RS485

Les produits de chez GCE Electronics ont une innovation que d'autres concurrents n'ont pas : une alimentation de 15 V via un câble RJ45 appelée EBX Connect/Power. Cette interface permet de facilement communiquer et d'alimenter tous les appareils compatibles.

Pour utiliser cette technologie, il est indispensable d'avoir un module RS485 qui permettra une communication dans le bus de données partagé avec jusqu'à 30 appareils. Pour contrôler ce module 3 connexions entre l'ESP32 Mesh et le module sont indispensables :

- DIR : Pour dire quand on a commencé et finis de communiquer

- TX : Pour transmission

- RX : Pour la Réception

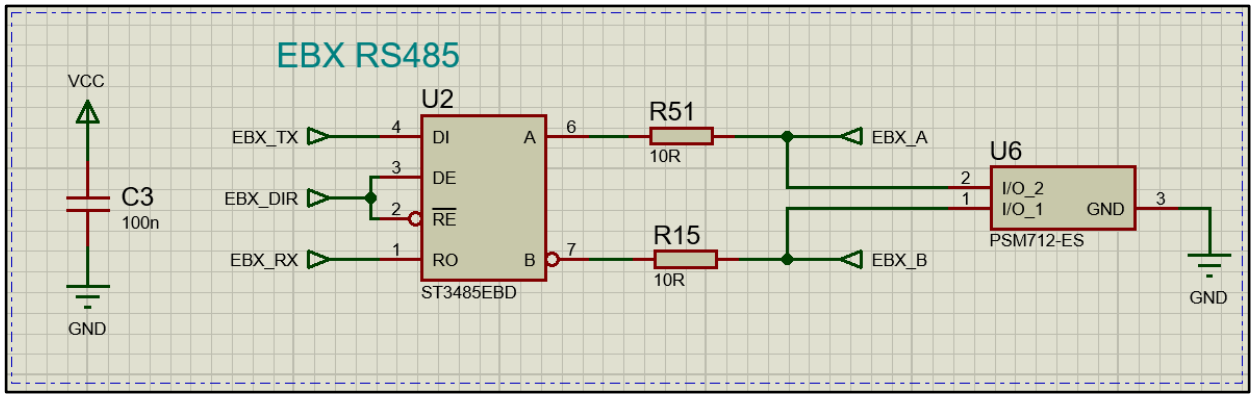


Figure 9 : Schéma électronique du module RS485

Lors de l'implémentation dans la carte finale, il a fallu être particulièrement précautionneux afin de respecter les principes de la CEM. Il était impératif de ne pas croiser les pistes, ce qui fut d'autant plus compliqué sachant que celles-ci traversaient une grande partie du PCB.

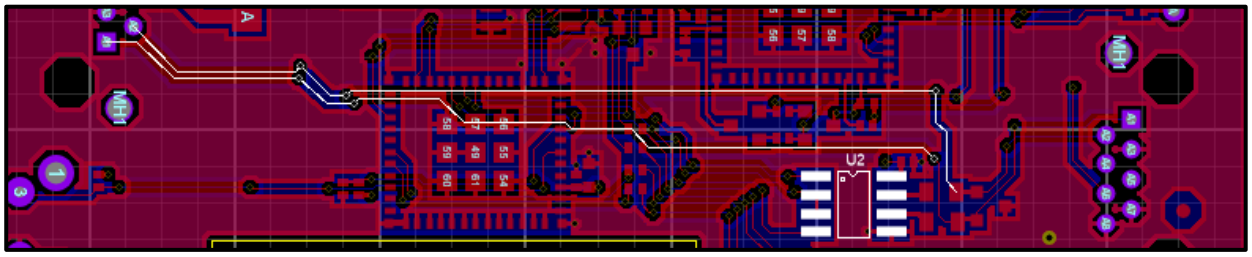


Figure 10 : PCB avec les pistes EBX A, B et le module RS485 surlignés

2.4 Les plus de la carte : l'EEPROM, les 2 LEDs et le bouton

Pour le produit final, il était absolument nécessaire de rajouter des LEDs pour notifier à l'utilisateur des informations importantes. La LED verte D2 s'occupera de notifier de l'alimentation ainsi que le mode de fonctionnement de l'ESP Mesh, tandis que la LED orange D1 s'allumera lorsque la carte est connectée au réseau Mesh.

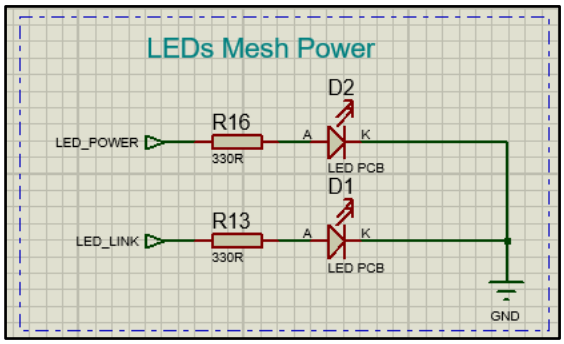


Figure 11 : Schéma électronique des LEDs

Par ailleurs, un bouton relié à l'ESP Mesh permettra de naviguer entre les modes de fonctionnement de l'ESP Mesh et BLE. Voici le schéma électronique du bouton de sélection de mode :

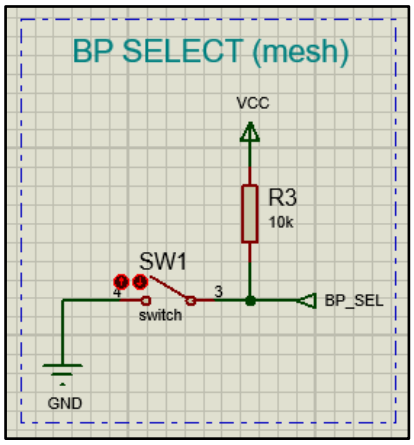


Figure 12 : Schémas du bouton de sélection de mode

Enfin, un module EEPROM a été ajouté, cependant il ne sera pas utilisé dans la suite du projet. En effet, Jérémy m'a expliqué qu'il pourrait servir pour une mise à jour afin de stocker les trames qui ont été corrompues et de savoir de quelle extension celle-ci provenait. Grâce au module, on pourrait également stocker quelle partie du réseau est davantage sollicitée (Mesh ou Connect) dans le but d'afficher des statistiques du réseau sur une interface web.

2.5 Les cerveaux de la carte

En ce qui concerne les 2 ESP32 (Mesh et BLE), comme expliqué précédemment leur schéma a été repris de la carte repeatDAC qui sera utilisé dans la suite du document pour la « proof of concept » de l'extension EBX-Link.

La partie réception Bluetooth, sera composée de la connexion UART (TX et RX) avec l'ESP Mesh, ainsi que son module d'auto-flash et de son connecteur permettant de reprogrammer L'ESP.

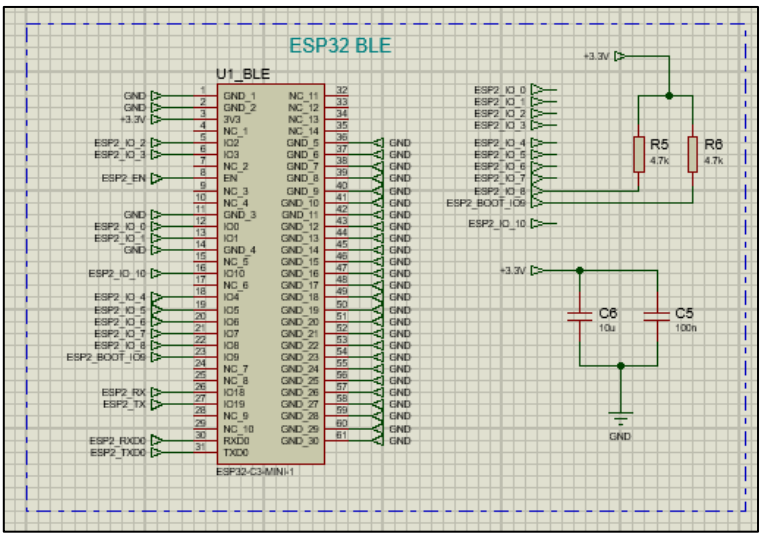


Figure 13 : Schéma électronique de l'ESP BLE

L'ESP Mesh reprendra entre autres la même base (Auto flash, UART …), à laquelle viendra s'ajouter le module RS485 pour la communication sur le BUS EBX, mais aussi les 2 LEDs, le bouton de sélection de modes et enfin l'EEPROM. En bref, l'ESP Mesh sera le véritable cœur de la future carte.

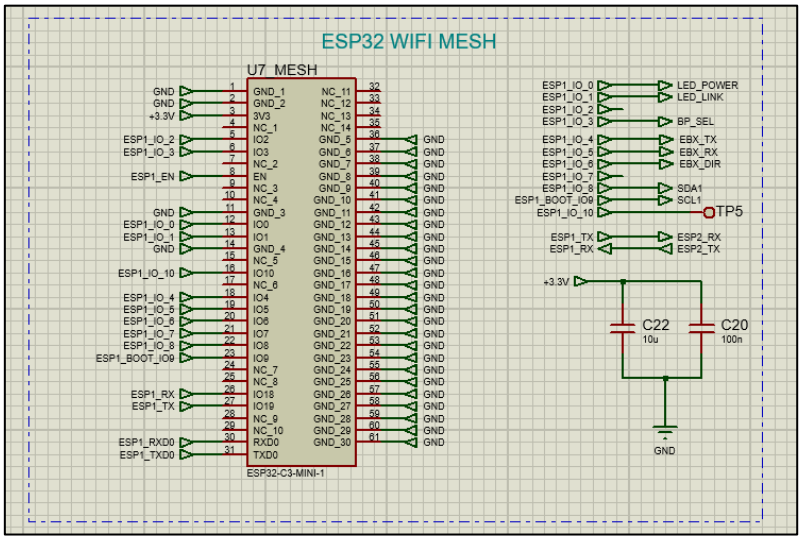


Figure 14 : Schéma électronique de l'ESP Mesh

À propos de leurs circuits d'auto-flash et leurs connecteurs, ils seront chargés de mettre automatiquement la puce dans le bon mode de fonctionnement afin qu'elle soit prête à recevoir les données du programme.

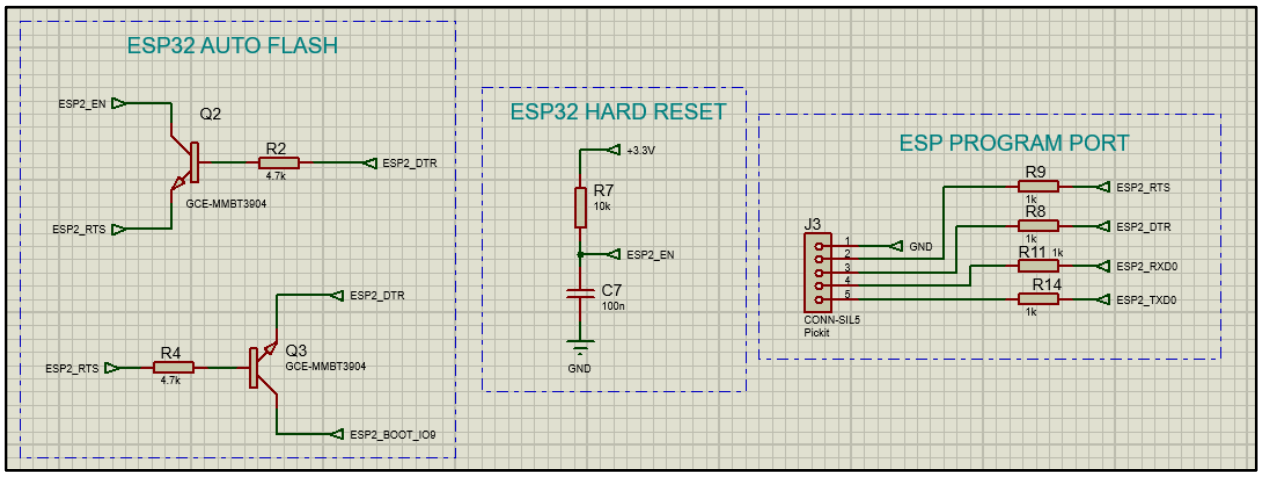


Figure 15: Schéma électronique du circuit d'auto-flash et de ses composants annexes

2.6 Les spécifications du PCB et le routage de celui-ci

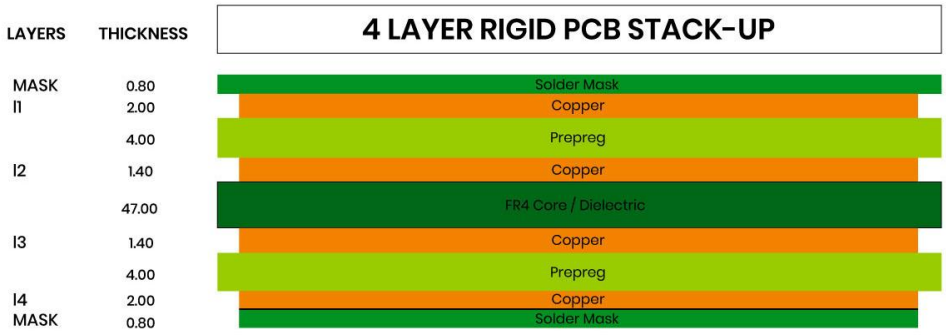


Figure 16 : Schéma interne d'un PCB 4 couches

Mon PCB étant limité à un design de 4 couches (voir les figures 16 et 17), celles-ci ont été organisées afin d'optimiser la CEM :

- Top : Signaux (plan de masse)
- Inner 1 : GND (plan de masse)
- Inner 2 : Alimentation (plan de VCC)
- Bottom : Signaux (plan de masse)

Cette configuration (stack-up) permet d'encapsuler l'alimentation entre deux plans de masse, ce qui est optimal pour la CEM. On remarque également la présence de nombreux vias, qui permettent d'optimiser le PCB en évitant de fractionner les plans de masse.

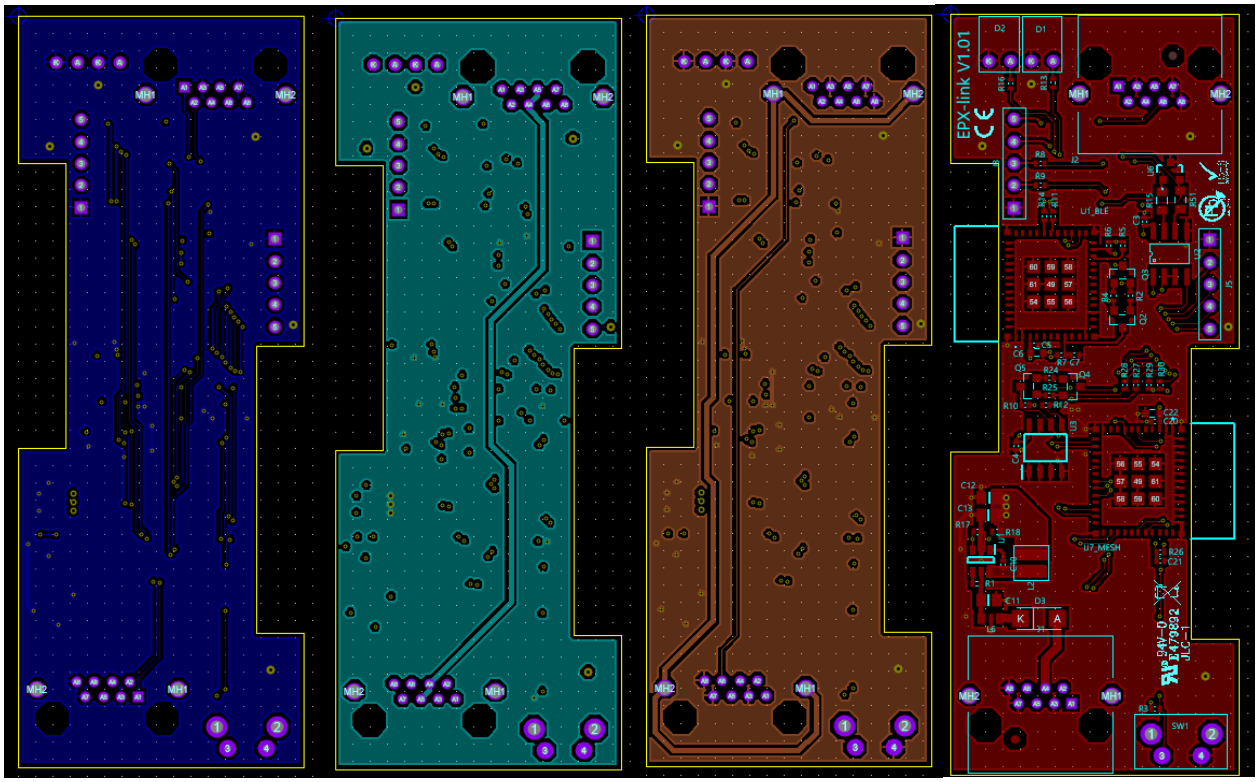


Figure 17 : Les 4 couches du PCB finale de l'EBX-Link

En outre, on peut remarquer que les pistes entre la couche « bottom » et la couche « top » sont perpendiculaires entre elles. Cela permet généralement de faciliter le routage et d'encore améliorer la CEM sur la carte en limitant les zones d'interférences.

Enfin, les couches internes sont peu fournies en pistes, mais elles possèdent les pistes les plus imposantes. Notamment, « l'inner 2 » qui gère l'alimentation 15 V de l'EBX-Power, tandis que « l'inner 1 » est utilisé pour la connexion earth entre les connecteurs RJ45. Cependant, ces couches sont denses en vias, permettant de relier chaque composant au GND ou au VCC (3.3V), ce qui facilite le routage et améliore la stabilité électrique de la carte.

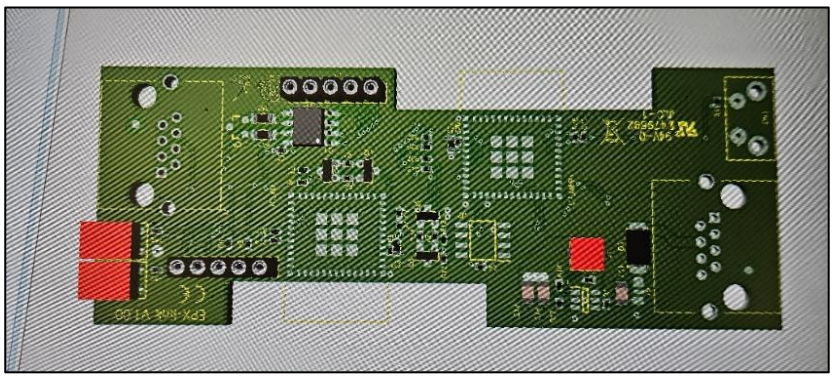


Figure 18 : Simulation 3D de la carte EBX-Link sur Proteus

2.7 Les bonnes pratiques pour un routage de PCB

Vérifier

Pour un bon routage de PCB respectant les règles CEM et d'intégrité du signal, il est essentiel d'adopter de bonnes pratiques afin de minimiser les interférences et les signaux parasites. Voici les principes que Patrick m'a transmis :

1. **Utilisation de pistes adaptées :** Lorsque l'espace le permet, il est préférable d'utiliser des pistes aussi larges que possible. Pour les signaux la taille minimale que j'ai utilisée était T6, mais j'ai pu monter jusqu'à T10 lorsque l'espace disponible le permettait.
   Pour les signaux de puissance, j'ai utilisé les tailles suivantes :
   - *T25 → 15V*
   - *T12 → 3.3V*
2. **Montage à deux condensateurs :** Un gros condensateur de stockage (bulk) lisse les variations de tension et fournit une réserve d'énergie. Tant dis que le condensateur de découplage filtre les bruits haute fréquence et les perturbations. Ils ont tous deux été placés en amont des composants critiques (EEPROM, ESP, module RS485…).

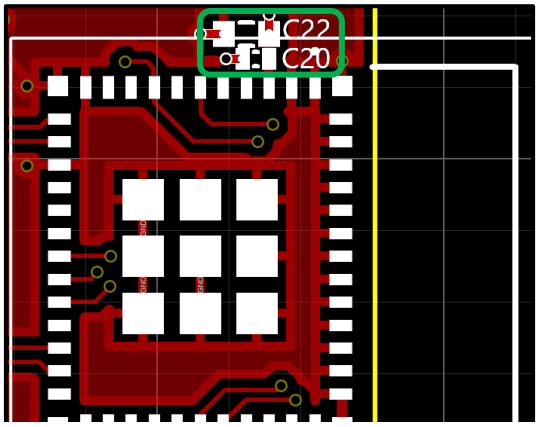


Figure 19 : Filtrage à 2 condensateurs sur les ESP

   De plus, on ajoute des vias reliés au GND le plus proche possible ce qui permet d'améliorer la dissipation des interférences (voir Schéma 1).

3. **Équilibrage du plan de masse :** Dans un PCB il faut limiter les coupures dans le plan de masse. Là où il y en restait, des vias ont été ajoutés afin d'équilibrer la masse et d'éviter les perturbations des signaux aux alentours dus à un effet d'antenne.

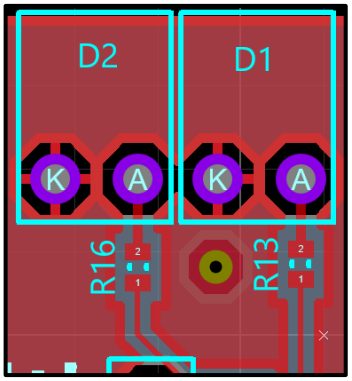


Figure 20 : Partie supérieure gauche du PCB, où un via permet d'équilibrer un plan de masse isolé

4. **Éviter de router en diagonale :** Dans un PCB, généralement le chemin le plus court est souvent en diagonal. Cependant, Il est préférable d'éviter ce genre de routage afin de réduire la surface de croisement des pistes. En effet plus les pistes se chevauche plus elles risquent de se perturber entre elles.

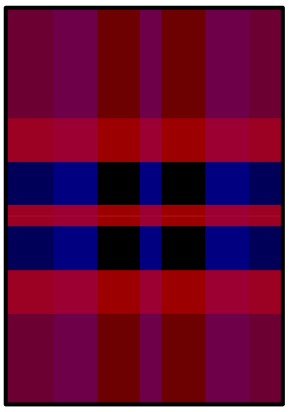


Figure 21 : Croisement court et optimisé

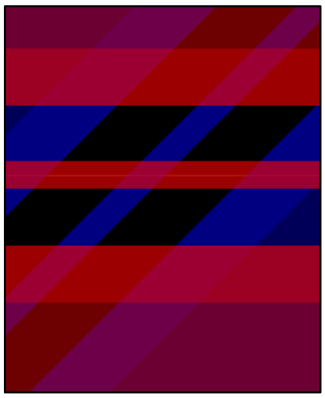


Figure 22 : Croisement long, non optimal

5. **Alignement et assemblage :** Enfin, l'ajout de fiducial permet d'aligner précisément le PCB sur la machine de placement automatique de composants, garantissant un assemblage fiable et précis.

Pour un projet avec des vitesses relativement lentes (115200 bauds) et ne nécessitant pas d'être ultra précis, toutes ces règles ne sont pas essentielles. Cependant, il est important de prendre de bons réflexes en conception électronique. De plus, un bon PCB tiendra mieux dans le temps, et aura moins de défauts de fabrication lors de la production.

2.8 Conclusion sur la conception

Avant d'aboutir à ce résultat final, j'ai dû en apprendre beaucoup sur la CEM et développer les compétences acquises au cours de ma formation. Notamment : concevoir une carte électronique dans son intégralité. Pour en apprendre plus, j'ai dû m'y reprendre à plusieurs reprises, recommencer les routages et optimiser le placement des composants, afin d'appliquer tous les concepts que j'ai appris au fur et à mesure. Il est difficile voire impossible de faire le PCB parfait dès le premier coup, l'esprit critique est indispensable afin d'avoir le meilleur PCB.

Pour finir, la production de cinq prototypes est une étape qui prend du temps. Avant de transmettre les fichiers au fabricant, il était donc impératif de valider la " proof of concept" afin de s'assurer que le hardware (les ESP32) pourra s'adapter aux exigences techniques.

3 - La proof of concept

La 2ème chose que j'ai réalisée chez GCE Electronics après avoir désigné la carte pour l'extension EBX-Link était de prouver la faisabilité du projet. En effet, en établissant le cahier des charges l'équipe avait un doute quant à la capacité d'un esp32 à pouvoir reconfigurer son UART de débogage. Pour démontrer la faisabilité du projet, la documentation technique de l'ESP32-C3-mini a été utilisée comme une référence principale.

3.1 PlateformIO et le Framework Arduino

Avant de commencer à coder, il est nécessaire de choisir le bon environnement de développement appelé IDE. Chez GCE Electronics, pour coder les ESP32, ils utilisent VS code avec l'extension PlateformIO afin de développer leur ESP32.

J'ai donc dû les installer sur la machine qui m'a été mise à disposition. Aussi, une spécificité des ESP32 c'est qu'on peut soit utiliser le Framework Arduino (langage C au niveau), mais il existe aussi le Framework ESP-IDF (langage C bas niveau), créé par espressif lui-même le fabricant de la puce ESP32-C3-mini utilisée. https://idf.espressif.com



Figure 23 : Logo de platformIO et Arduino IDE

Une fois sa plateforme de développement, on peut configurer son premier projet. Pour configurer son projet il suffit de modifier le fichier plateformio.ini et plateformIO s'occupe d'installer et d'initialiser tout ce dont il a besoin pour fonctionner. Pour le projet EBX-Link PlatformIO, a été configuré de cette manière :

```ini
platformio.ini
1
2   [env:esp32-c3-devkitm-1]
3   platform = espressif32
4   board = esp32-c3-devkitm-1
5   framework = arduino
6   upload_protocol = esptool
7   upload_port = COM5  ; Remplacez COM15 par le port série approprié
8   upload_speed = 115200 ; attention pas de vitesse trop lente (9600)
9   monitor_speed = 115200  ; Vitesse pour le moniteur série
```

Figure 24 : Extrait de la configuration de plateformIO dans Visual Studio Code

- **Plateforme :** espressif32, qui correspond aux microcontrôleurs ESP32 de la marque espressif.
- **Board :** esp32-c3-devkitm-1, qui est le kit de développement correspondant à la spécifique carte utilisée ESP32-C3-MINI.
- **Framework :** Arduino, ce qui permet de programmer l'ESP32 avec l'environnement Arduino.
- **Protocole de téléversement :** Esptool est le protocole utilisé afin de flasher le microcontrôleur.
- **Vitesse de téléversement et du moniteur série :** 115200 bauds pour une communication rapide et stable, avec vitesse plus faible, j'ai eu des problèmes de téléversements.

3.2 Debugger et téléversement, le premier programme

Concevoir

Après avoir téléchargé et mis en place l'interface de développement, la prochaine étape consiste à téléverser le premier programme vers l'ESP. Pour cela, un premier programme de test visant à vérifier les principales fonctionnalités du microcontrôleur.

J'ai opté pour une solution simple : des LEDs qui clignotent. Un classique, mais efficace ! J'ai également ajouté un 'Hello world' dans la liaison série. Cela permettra de vérifier si PlateformIO est bien configuré et que les GPIO de l'ESP sont correctement attribués aux LEDs.

Une fois ce premier programme créé, il faut le téléverser. Dans un premier temps, j'ai tenté d'utiliser un ESP-Prog pour le téléversement. Cependant, comme mentionné précédemment, la carte disposait déjà d'un circuit d'auto-flash, tout comme l'ESP-Prog. Cela empêchait l'ESP d'entrer correctement en mode programmation et de recevoir le code.

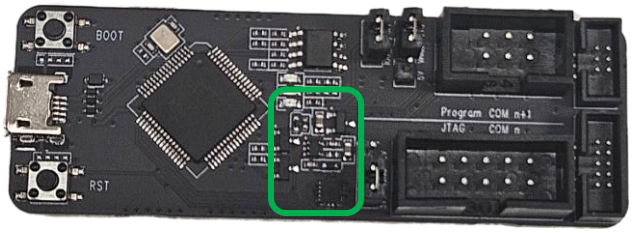


Figure 25 : ESP-prog, avec son circuit d'auto-flash entouré en vert

Heureusement, dans un second temps, Kevin m'a donné une carte qu'il a conçue lui-même, compatible avec le protocole esptool, et sur laquelle on peut activer ou désactiver le circuit d'auto-flash via des jumpers. Grâce à celle-ci, le RepeaterDAC a pu être reprogrammé avec le programme de tests.

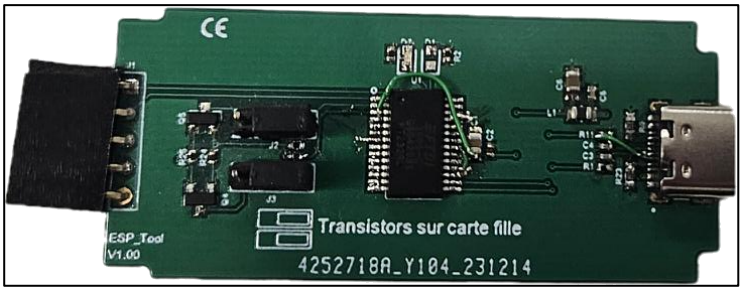


Figure 26 : Programmer ESP Tool fabriqué par GCE

3.3 Vérification du premier programme de test

Vérifier

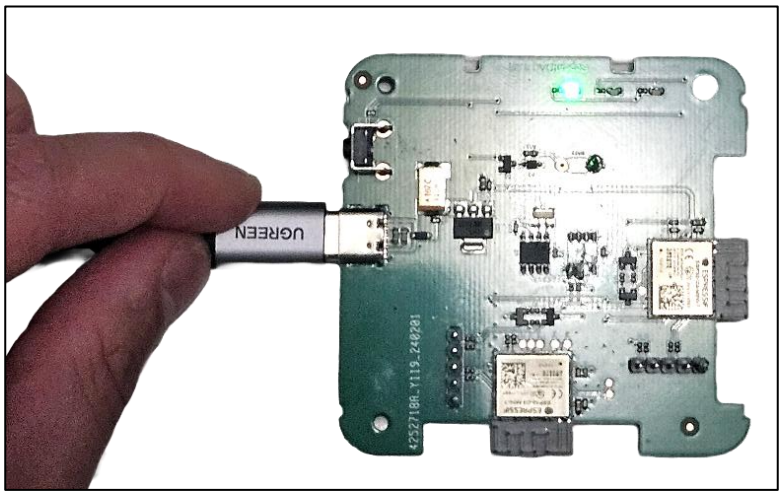


Figure 27 : Carte repeatDAC, programmée avec le programme de tests

Une fois la carte alimentée via son port USB-C et correctement programmée. La LED power de la carte clignote. De plus, une réponse de la carte est bien reçue dans le moniteur série, confirmant le bon fonctionnement de la communication série et de la bonne exécution du programme.

```
test liaison serie : OK<CR><LF>
```

Figure 28 : Réception de la liaison série avec l'ESP

3.4 Développement informatique de la «proof of concept» Concevoir

Maintenant, pour vérifier la faisabilité de notre carte, il faut ajouter au programme une autre liaison UART, pour cela, les 2 ESP (MESH et BLE) ont été reprogrammés (Figures 27 et 28).

ESP Mesh : Lorsque l'ESP Mesh démarre, il configure ses ports pour les LEDs (OUTPUT), le bouton (INPUT) ainsi que les 2 liaisons UART. Après il vérifie, dans la boucle principale, si le bouton a été activé : dans ce cas il envoie OK dans l'UART 0. Et il vérifie également s'il a reçu le caractère « 1 » dans l'UART 0 : Si c'est le cas, il envoie test de liaison série : OK sur l'UART 1.

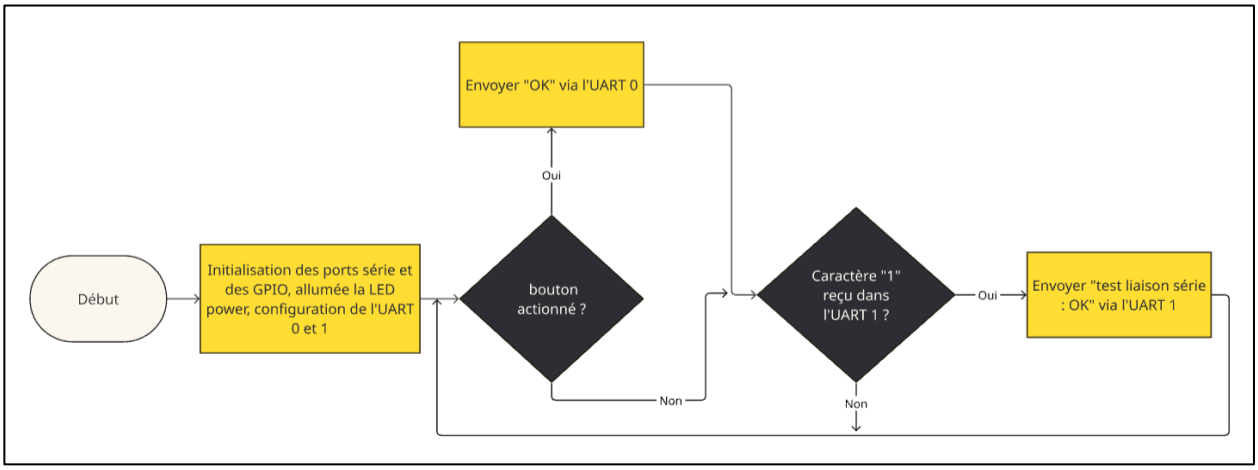


Figure 29 : Diagramme de flux du programme de proof of concepts (Partie ESP Mesh)

ESP BLE : Quand l'ESP BLE démarre il initialise son UART 0, puis entre dans la boucle principale : s'il reçoit quelque chose de cette liaison alors il envoie le caractère « 1 ».

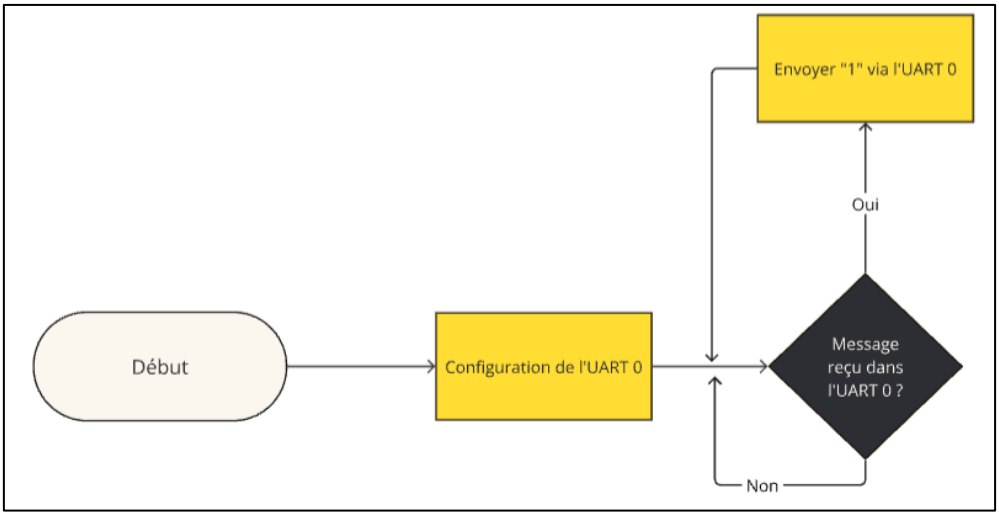


Figure 30 : Diagramme de flux du programme de proof of concepts (Partie ESP BLE)

Le code C ci-dessous permet de reconfigurer simplement les UART en utilisant le Framework Arduino. Pour faire simple, l'UART 0 (UART de debug) de l'ESP Mesh est assigné aux connexions RX et TX entre l'ESP BLE et Mesh. L'UART 1 est quant à lui redirigé vers les GPIO 8 et 9, qui n'étaient pas utilisés sur la carte du repeatDAC.

```
HardwareSerial mySerial(0);
HardwareSerial mySerial1(1);
mySerial.begin (115200, SERIAL_8N1, 18, 19 );
mySerial1.begin (115200, SERIAL_8N1, 8, 9 );
```

Figure 31 : Configuration des liaisons série de l'ESP Mesh

Afin de vérifier le bon fonctionnement du programme, la carte repeatDAC a quand même subi quelques modifications. En effet, afin de pouvoir relier cette 2ème UART à un ordinateur j'ai dû rajouter des fils que j'ai reliés à un câble USB vers TTL UART.



Figure 32 : Câble USB vers TTL UART

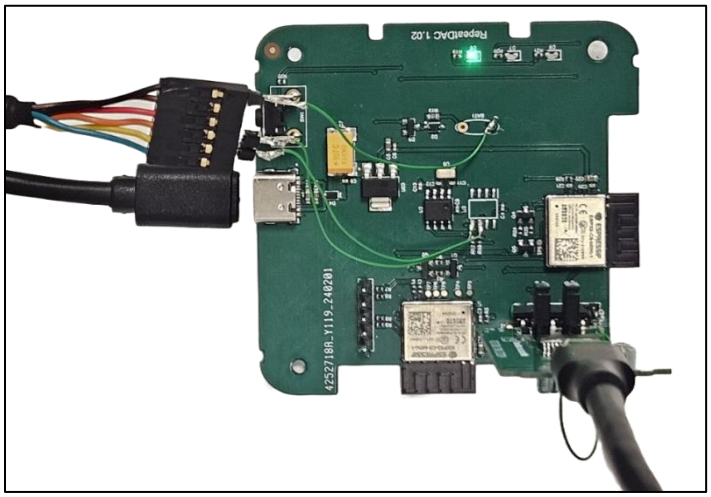


Figure 33 : RepeatDAC avec fils pour la connexion UART supplémentaire

3.5 Validation de la « proof of concept » et Conclusion

Vérifier

Une fois le montage effectué, la proof of concept a pu être présentée à Jérémy, et dès lors qu'elle a été validée, la commande des cartes de l'EBX-Link a été lancée avec Patrick. Pour commander ces cartes, a dû être créé le répertoire de production contenant tous les fichiers des différentes couches du PCB, permettant au fabricant de le concevoir. Dans, cette phase, on peut s'apercevoir que les PCB peuvent encore être optimisés en supprimant le circuit d'auto flash des cartes, cela permettrait de réaliser des économies sur les composants utilisés et de simplifier les cartes. (Cela est en cours de développement pour les prochaines cartes de l'entreprise)

4 - Le début du développement

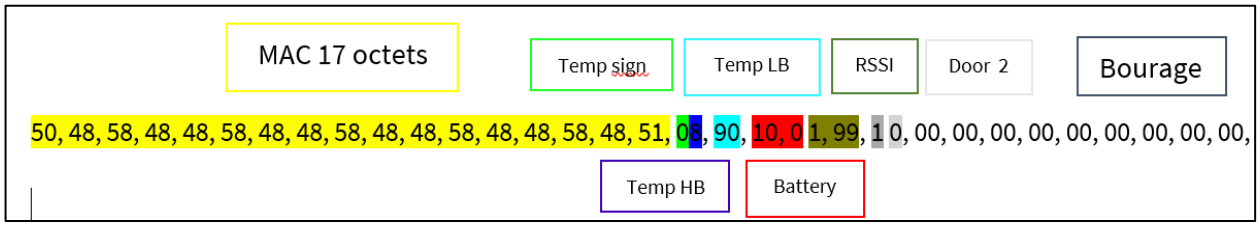
Cette phase était cruciale pour le développement de la carte finale, C'est à ce moment-là que je me suis pleinement imprégné de la base de code du RepeaterDAC, sur laquelle est basée une partie de mon hardware. Les débuts furent compliqués pour comprendre chaque partie du programme mais grâce aux explications claires de Jérémy, j'ai pu me familiariser avec les programmes rapidement.

4.1 L'UART de ESP BLE

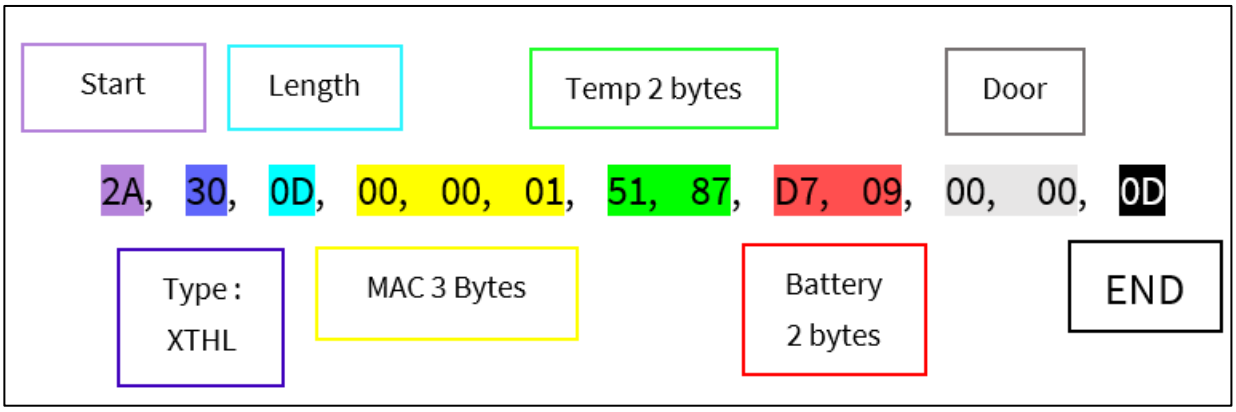
Concevoir

Pour la transmission des données entre l'ESP BLE et l'ESP Mesh, j'ai travaillé sur la partie du code permettant de convertir les données reçues de capteurs BLE vers l'ESP Mesh. Les valeurs des capteurs (température, batterie, et adresse MAC) étaient déjà automatiquement stockées dans un buffer. Mon objectif était alors de développer un programme qui convertie cette trame Bluetooth stockée, afin de forger une trame EBX (pseudo-code détaillé en annexe 1). Qui pourra notamment être interprétée par l'IPX 800 V5.

Trame reçue des capteurs Bluetooth (miniDAC) :



Trame EBX (X-THL) :



En ce qui concerne la réception, cela était déjà développé pour les tests de l'ESP Mesh, le programme permettant de communiquer via l'UART TTL a été réutilisé afin de visualiser la trame créée et de vérifier si celle-ci est correcte. Cela, m'a permis d'isoler cette partie du programme sans me soucier de la partie Mesh et donc d'être très efficace.

4.2 Capteur de température et réception sur l'IPX 800

Pour terminer, afin d'augmenter la fréquence des trames envoyées par les capteurs de températures, j'ai dû changer leur fréquence d'envoi de toutes les 10 minutes à toutes les 2 secondes pour les premiers tests puis à 30 secondes pour des tests plus réalistes. Pour ce faire, j'ai dû prendre en main l'IDE MPXLAB (environnement de développement intégré de Microchip), car les capteurs de température miniDAC sont dotés d'une puce PIC qui ont l'avantage de consommer peu d'énergie et donc dans notre cas un minium de pile.

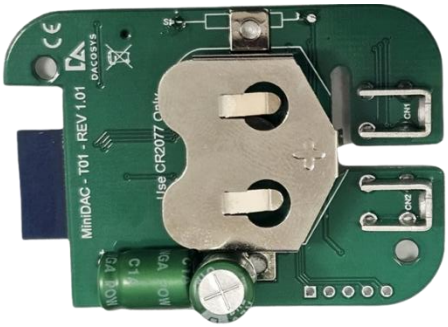


Figure 34 : Capteur de température (MiniDAC)

Afin d'alimenter la carte durant les tests, pour ne pas utiliser de pile on peut utiliser : le PICKit 4 (debugger de chez Microchip) en activant l'alimentation 3.3V (LED orange), ou on peut également utiliser une alimentation externe réglée sur 3.3V 0.1A pour se passer de la contrainte de la connexion avec un ordinateur.



Figure 36 : PICKit 4 de Chez Microchip

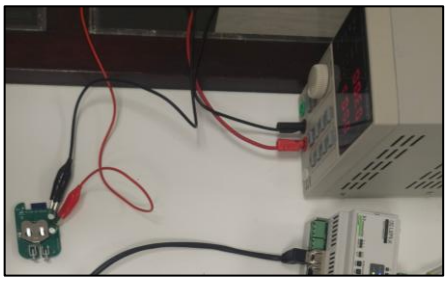


Figure 35 : Alimentation externe

Pour conclure, cette trame, une fois reçue par l'IPX, permettra d'afficher sur son interface Web la température et le niveau de batterie du capteur. (Voir annexe 2). Avant cela, la trame devra être relayée par l'ESP Mesh (partie 6.1).

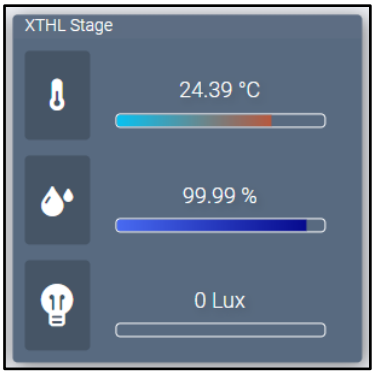


Figure 37 : Affichage de la température sur l'interface web de l'IPX 800 V5

5 - Réception du PCB et réalisation de prototype

Durant le développement logiciel, il a fallu faire une petite interruption. En effet, les premières cartes prototypes ont été réceptionnées. Dans le but de ne pas bloquer la production des autres cartes GCE Electronics, j'ai été amené à réaliser toutes les étapes de fabrication du PCB par moi-même. Cela a été très enrichissant pour moi, car en raison de la taille des composants, j'ai été amené à utiliser de nouveaux procédés de fabrication.

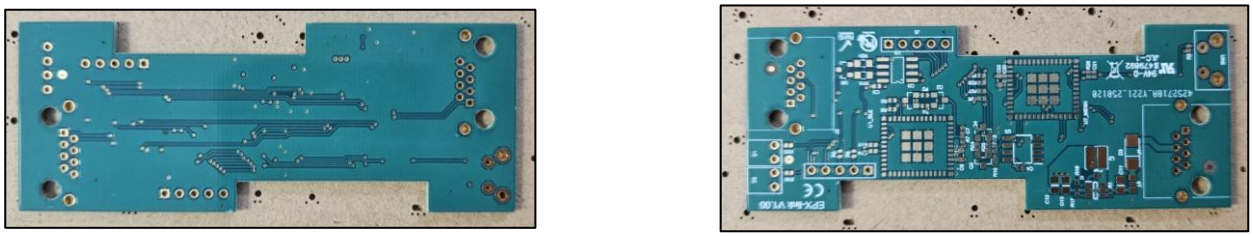


Figure 39 : Face avant et arrière de la carte EBX-Link



Figure 38 : Stencil découpé pour la carte EBX-Link

5.1 Sérigraphie et Pâte à braser

Concevoir

Après avoir reçue les cartes et s'assurer que tout est conforme, il est temps de passer à l'étape suivante : la sérigraphie et l'application de la pâte à braser. Cette étape permet de préparer la carte au placement et à la soudure des composants. Voici un guide étape par étape pour réaliser cette opération correctement :

1. **Découper le stencil :** Celui-ci est trop grand pour rentrer dans la machine il faut donc le découper. (Figure 33 ci-dessus)
2. **Aligner le PCB :** Pour l'aligner, plusieurs molettes de réglage sont disponibles, pour permettre à la pâte à braser de s'appliquer correctement



Figure 40 : Stencil dans la machine de sérigraphie manuelle

3. **Bien étaler la pâte :** Il faut bien appuyer avec le racloir pour que la pâte s'étale uniformément sur les pins de la carte électronique.

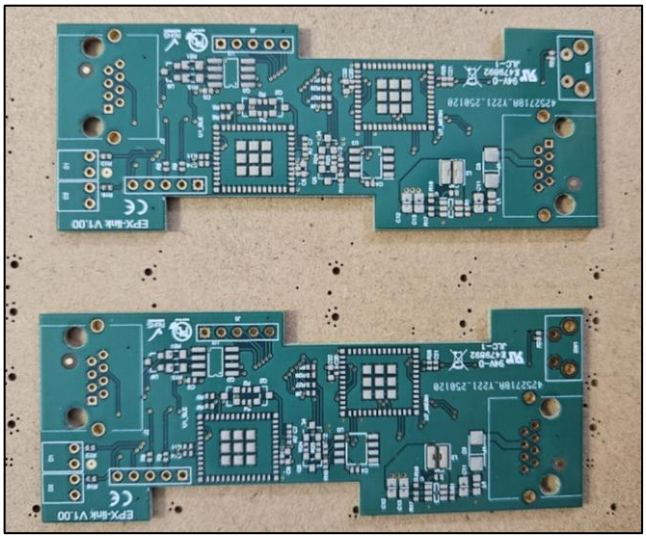


Figure 42 : PCB sérigraphiés



Figure 41: Racloir induit de pâte à brasée

5.2 Placement minutieux de chaque composant

C'est de loin la phase la plus critique du processus de fabrication manuelle des PCB. Il existe différentes méthodes en ce qui concerne l'ordre du placement des composants (petit≠gros), mais j'ai opté celle-ci : placer d'abord les composants, les plus petits, quand ils sont isolés et placer d'abord les composants les plus gros quand les petits sont proches. Cette approche a été la plus efficace pour moi, après avoir créé quatre cartes différentes. Pour garantir la précision, une binoculaire et une pince précelle sont utilisées afin d'être le plus minutieux possible.



Figure 44 : Binoculaire

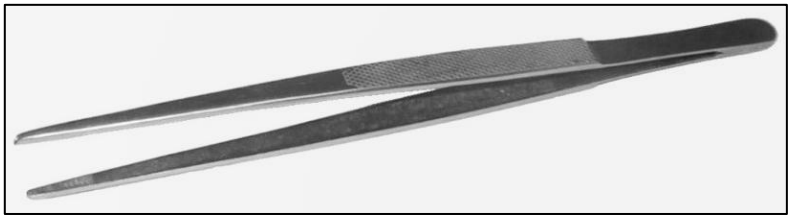


Figure 43 : Pince precelle

5.3 Four à refusion

Après avoir placé les composants de manière optimale, il est temps de les passer au four à refusion pour faire fusionnés les soudures. Cette étape ne diffère pas de la méthode industrielle.

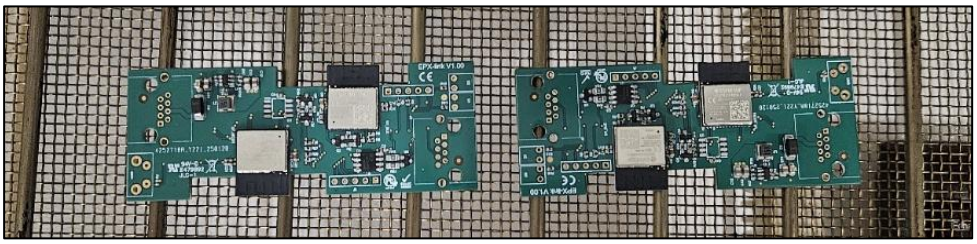


Figure 45 : EBX-Link qui rentre dans le four à refusion

Enfin, pour terminer la fabrication des PCB, il ne manque plus qu'à souder les composants traversants un à un afin de compléter la carte électronique.

A ce stade, j'ai remarqué que l'empreinte des boutons avait été placée à l'envers sur la carte (Carré orange figure 41), alors je l'ai remise à l'endroit sur Proteus pour les futures cartes produites en série.

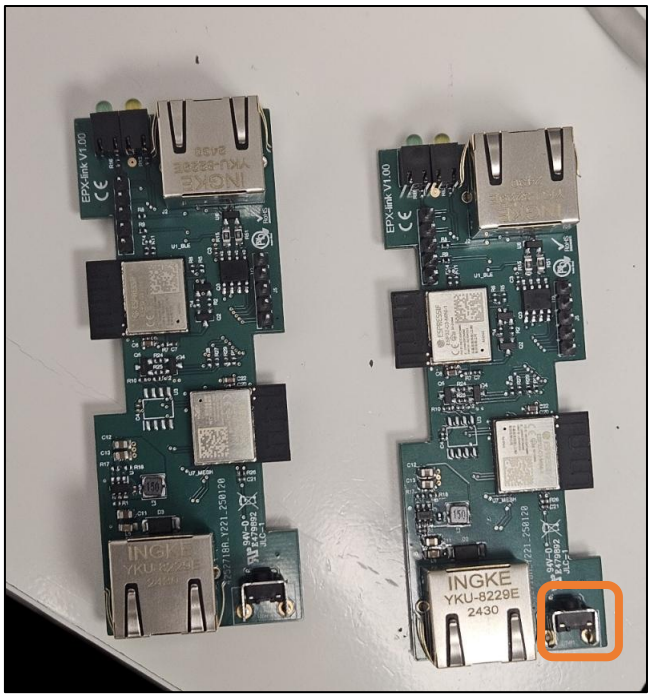


Figure 46 : Prototype de l'EBX-Link, entièrement fabriqué

5.4 Vérification et validation de la carte EBX-Link

Vérifier

Il peut arriver que certains composants se retrouvent à la verticale à cause d'un mauvais contact avec la pâte à braser, un phénomène appelé "effet Manhattan" ou "effet Tombstone".

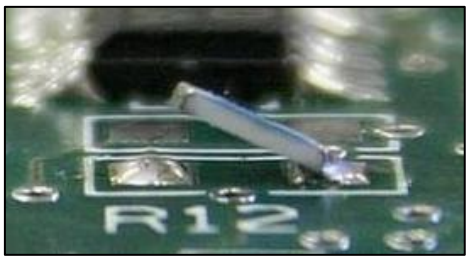


Figure 47 : Résistance ayant été affectée par l'effet Tombstone

Dans ce cas, il est nécessaire de ressouder la résistance à la main, ce qui peut être difficile avec des composants très petits (comme les 0402). De plus, les résistances étant petites, il est facile de se tromper de valeur, j'ai pris pour habitude de vérifier leur valeur avec un multimètre en cas de doute.

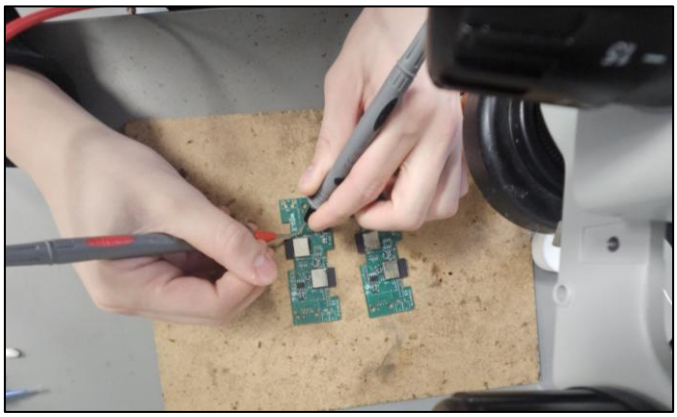


Figure 48 : Moi-même contrôlant les valeurs des résistances de la carte

Enfin, parfois, certains composants peuvent être mal soudés, cela m'est arrivé avec mon premier prototype où j'ai dû remplacer un ESP32 en utilisant une station à air chaud.

Produire à la main quelques prototypes peut être viable, mais il est impensable de nos jours de produire de gros volumes sans passer par des machines. C'est pourquoi GCE Electronics a investi dans des machines de qualité permettant d'optimiser leur processus et ainsi de créer un maximum de cartes durables. (Voir annexe 6)

6 - La suite du développement logiciel

Une fois la conception du prototype terminée, le développement logiciel a continué.

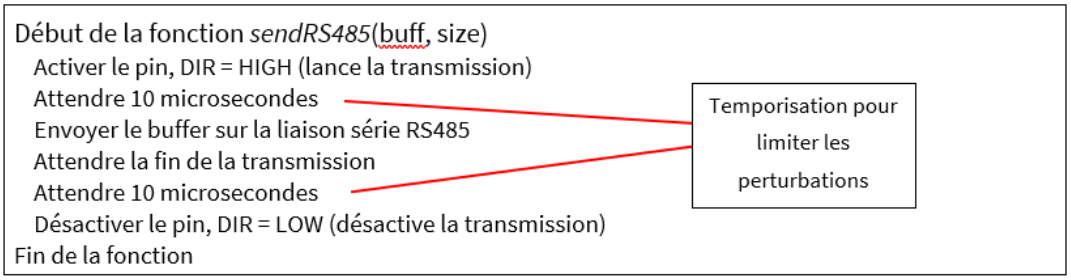
6.1 BLE vers EBX MESH et EBX Connect

Concevoir

Une fois la partie BLE développée, la suite logique était le développement de la réception de la trame par le programme de l'ESP Mesh et de transférer cette trame vers le réseau EBX Mesh et l'EBX Connect (RJ45).

Je partais d'une base solide pour le côté EBX Mesh avec la bibliothèque (painlessmesh), qui permet de gérer efficacement le réseau Mesh. De plus, la récupération de la trame BLE via l'UART était déjà développée.

Afin de mener à bien le projet, il fallait renvoyer vers les deux réseaux (EBX Mesh et EBX Connect) la trame reçue. C'est à ce moment-là que la carte EBX-Link prend son rôle principal de passerelle entre d'un côté le réseau Bluetooth et les réseaux EBX Mesh (sans fil) et EBX Connect (filaire RJ45).
La première fonction qui a été programmée est la transmission vers l'interface EBX Connect et le module RS485. Celle-ci permet d'envoyer un buffer directement vers le module RS485 via l'UART 0.



En outre, pour vérifier si une nouvelle trame est reçue la fonction *UART_BLE_GET* est appelée, qui retourne vrai si une nouvelle trame est arrivée ou false si aucune nouvelle a été reçue. Si une trame est reçue alors on peut l'envoyer directement dans le réseau EBX Connect. Pour l'envoyer en Mesh, il y a un peu plus d'étapes, on la convertit d'abord en variable « String » puis on l'envoie en broadcast à tous les appareils du réseau Mesh grâce à une fonction déjà développée *sendMesage*.

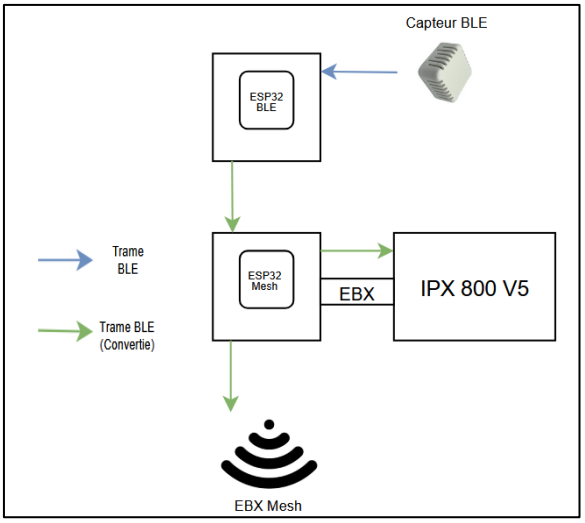


Figure 49 : Schéma des interconnexions entre les réseaux BLE et EBX

6.2 Du réseau Connect vers le réseau Mesh et inversement

Afin que l'extension EBX-Link puisse fonctionner comme une passerelle complète entre les deux réseaux (filaire et Mesh), il est nécessaire d'assurer une transmission efficace des trames entre ces deux environnements.

En reprenant ce qui avait déjà été développé pour la partie précédente, une solution a été mise en place permettant d'établir le lien entre le réseau filaire vers Mesh. Pour cela, dans la boucle principale la fonction UART_RS485 vérifie en permanence si des trames sont reçues et les stocke dans un buffer circulair. Grâce à la fonction UART_RS485_GET on peut détecter si une nouvelle trame a été reçue et la récupérer. Si une trame est détectée et que celle-ci commence bien par 0x2A (Bit de start), elle est transmise sur le réseau Mesh via la fonction vue précédemment : *sendMessage*.

Dans le cas inverse où on cherche à ce que L'EBX Link redirige les trames Mesh vers le réseau filaire EBX Connect :
On passe alors cette fois-ci par la « callback » de la bibliothèque. Si un message est reçu, alors la fonction *receivedCallback* est exécutée automatiquement, peu importe ce que fait l'ESP. Dans cette « callback » on reçoit une variable « string » MSG qui contient la trame reçue. Puis cette variable est convertie en octets que l'on met dans un buffer, enfin une vérification a été ajoutée avec memcmp :

```
if (memcmp(log,prefixEBX,1) == 0)
```

qui permet de vérifier si le premier octet du tableau log est égal au premier octet du tableau prefixEBX (0x2A). Cela permet d'enlever toute trame inutile ou corrompue qui pourrait se trouver sur le réseau. Si la trame commence bien par 0x2A alors elle est envoyée via le module RS485, dans le bus EBX (filaire RJ45).

Pour conclure, ces 2 parties sont essentielles afin de permettre à la carte EBX-Link de faire passerelle entre le réseau EBX sans fil et le réseau EBX filaire :

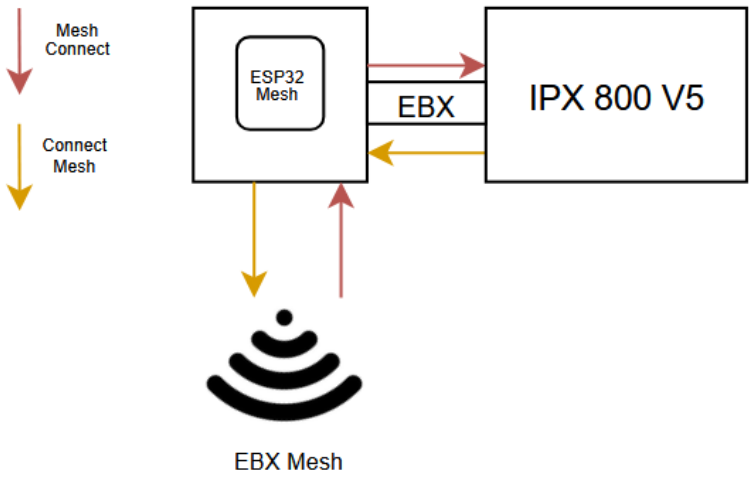


Figure 50 : Schéma des interconnexions entre les réseaux Mesh et Connect de la carte EBX-Link

6.3 Les fonctions supplémentaires

Avec Jérémy on a identifié un problème si 2 extensions EBX-Link sont connectées par un câble et sont sur le même réseau Mesh alors cela va créer une boucle infinie ! En effet, le message est envoyé en Mesh puis reçu et renvoyé en EBX Connect avant d'être encore envoyé…Saturant alors tout le réseau.

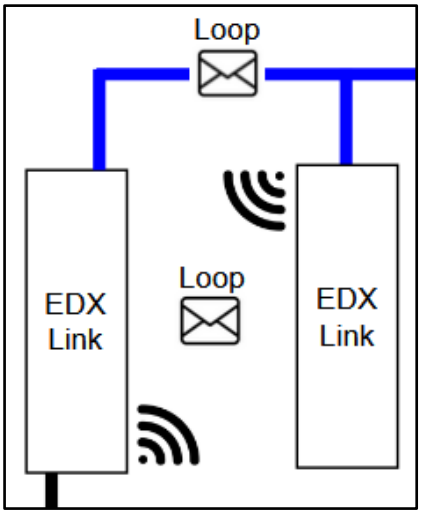


Figure 51: Schéma de 2 extensions EBX-Link qui bouclent entre elles

Pour se protéger de cela a été mis en place un système de détection de bouclage. Cela repose sur une méthode simple, toutes les 5 secondes une trame EBX dédiée est envoyée sur le réseau Mesh, si on détecte cette même trame dans l'entrée UART EBX Connect (Filaire) alors cela signifie qu'il y a eu un bouclage. Dans ce cas on redémarre l'ESP en mode de fonctionnement sleep où celui-ci n'interagit plus avec le réseau Mesh mettant alors fin à la boucle infinie.

```
#define LOOP_DETECTION_MSG  "2AAF04000000EE0D"
```

Figure 52 : Trame de détection de boucle

Par la suite on ajoutera une fonctionnalité permettant de changer le mot de passe et le SSID du réseau Mesh via l'interface WEB. Pour cela, on enverra une trame à tous les appareils dont on souhaite modifier leur SSID et mot de passe (Figure 47). Si on détecte ce type de trame dans la « callback » alors la fonction *processPrivateSSID_PASS* permet de redémarrer l'ESP avec le nouveau mot de passe et SSID du réseau Mesh.

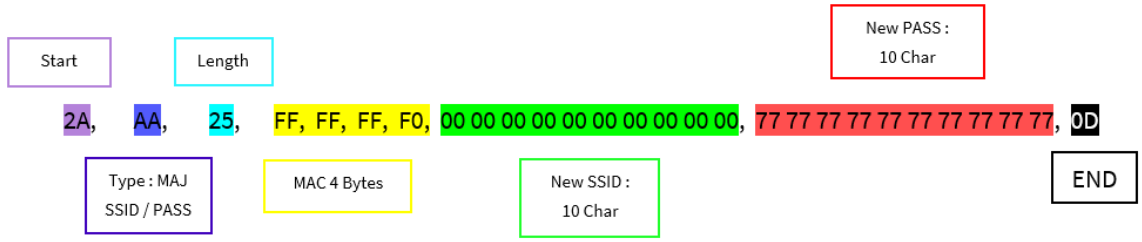


Figure 53: Trame EBX de mise à jour du réseau Mesh (SSID et Password)

6.4 Mise en mémoire flash avec un ESP

Comme expliqué, pour le moment le module EEPROM n'est pas utilisé. Cependant, il est essentiel de stocker plusieurs informations afin de redémarrer proprement l'ESP lorsqu'il passe d'un mode de fonctionnement à un autre. Pour cela, la bibliothèque Storage a été utilisée, elle permet de stocker directement dans la mémoire Flash de l'ESP Mesh des variables. Pour cela, voici les 2 fonctions que j'ai utilisées :

- `void Storage_put(String key, uint8_t size, uint8_t *buff)` Afin de mettre en mémoire.
- `uint8_t Storage_Get(String key, uint8_t size, uint8_t *buff)` Pour récupérer depuis la mémoire.

A l'aide de la première commande j'ai créé une fonction qui permet d'initialiser les variables stockées (avec *Storage_Get*) en mémoire lors du démarrage de l'ESP. Cette fonction va récupérer les valeurs enregistrées en mémoire et elles seront utilisées pour déterminer le mode de fonctionnement principal de chaque microcontrôleur, ainsi que les paramètres spécifiques aux sections du programme, comme les informations de connexion Mesh (SSID et mot de passe).

6.5 Changement de mode sur l'ESP Mesh

C'est sur l'ESP Mesh que repose une fonction permettant le changement de mode de fonctionnement des ESP. Cette fonction (*BP_mode_switch*) permet de modifier et d'afficher le mode de fonctionnement actuel. Il s'agit de l'une des fonctions principales du programme, car elle s'exécutera en arrière-plan dans la boucle principale du code. Elle se doit d'être particulièrement optimisée afin de ne pas créer de latence ou de retard sur les fonctions plus spécifiques de la carte.

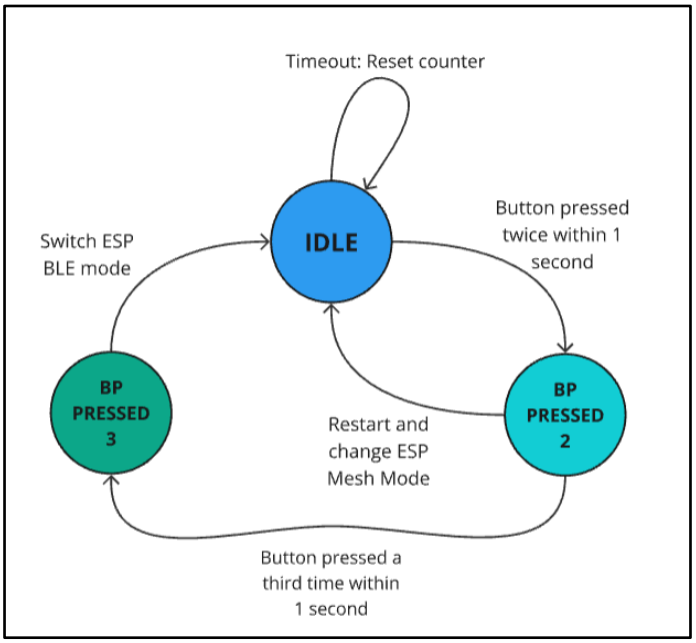


Figure 54 : Machine à état représentant les changements de mode gérés sur l'ESP Mesh

Concernant les LEDs, cela est géré directement dans les boucles principales des modes. Par exemple, en mode OTA, j'ai développé une fonction permettant à la LED verte power de clignoter. En mode Mesh, si le Mesh est connecté, alors la LED orange Mesh s'allume sinon elle est éteinte. Si l'ESP est en mode sleep (Mesh désactivé), alors il n'y a que la LED verte qui s'allume (figure 49). Un point d'amélioration aurait été d'avoir plus de LEDs ou des LED RGB, car actuellement l'utilisateur pourrait confondre un mode sleep avec le mode Mesh sans connexion.

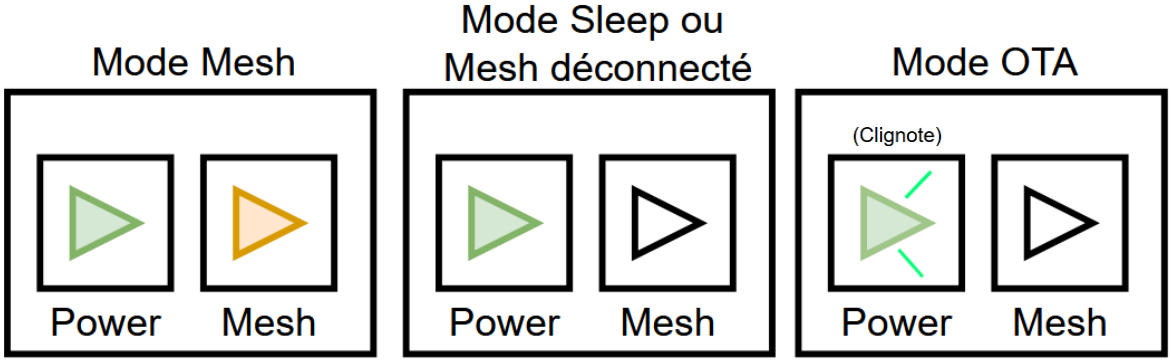


Figure 55 : Schéma représentant l'état des LEDs en fonction des différents modes de fonctionnement

7 - Le développement WEB

D'ans un autre contexte, il était nécessaire de mettre en place un moyen de mettre à jour facilement les ESP. Cela permettra de faciliter la maintenance et de, par la suite, ajouter de nouvelles fonctionnalités et régler des bugs lorsqu'ils sont trouvés.

7.1 Partie WEB mise à jour OTA

Maintenir

Pour l'ESP BLE, dans le code repris du repeaterDAC, il y avait déjà d'implémenter un système permettant de mettre à jour simplement le microcontrôleur.

Une bibliothèque nommée asyncElegant OTA, permet d'accéder à une page web hébergé sur l'ESP via un réseau wifi spécifique actif dans ce mode. La page web permet de téléverser un fichier de mise à jour de l'ESP (figure 49).

Afin d'entrer dans ce mode, il faut appuyer 3 fois sur le bouton principal de l'ESP Mesh. Celui-ci envoie dans la liaison UART reliant les deux contrôleurs les caractères : " OTA " à l'ESP BLE. J'ai donc ajouté un système permettant de détecter si les caractères 'OTA' sont reçus par l'ESP BLE. Si ces caractères spécifiques sont reçus, alors l'ESP change de mode et passe en mode mise à jour wifi.
Enfin, cette page était personnalisée avec le logo et les couleurs de GCE électronique.

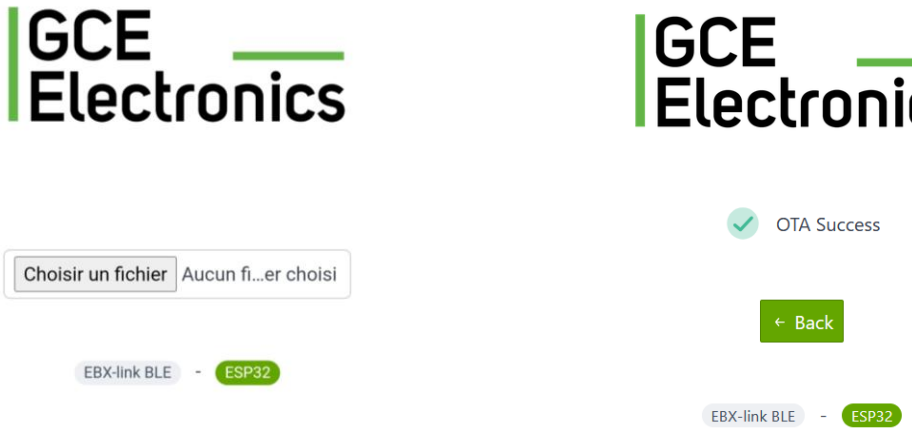


Figure 56 : Interface WEB de mise à jour de l'ESP BLE

7.2 Le site WEB embarqué de l'ESP Mesh

Pour l'ESP Mesh, je devais aussi mettre en place un site auto hébergé par L'ESP cependant, la librairie asyncElegant OTA stock la totalité du site web embarqué en base64(plus de détails dans l'annexe 4). Ce qui rendait difficile sa modification. Et justement, pour cette partie je ne devais pas simplement mettre un système de mise à jour mais aussi un système permettant la configuration du réseau Mesh.

Cette fonctionnalité de configuration que j'ai développée permet de changer le mot de passe et le SSID du réseau mais aussi de le désactiver entièrement, si l'utilisateur le souhaite.

Je me suis d'abord occupé de tout ce qui concerne l'interface utilisateur afin qu'elle soit intuitive et réponde au besoin de simplification pour le grand public, en ajoutant tous les boutons et formulaires (voir figure 50 ci-dessous)

J'ai par la suite appris à récupérer le mot de passe, le statut privé ou public et l'état du Wi-Fi via un formulaire utilisant la méthode POST, c'est-à-dire hors de l'URL, afin de sécuriser les informations de connexion.

Une fois récupérés, le nouveau mot de passe et le SSID sont stockés en mémoire. Puis, l'ESP entre dans une phase d'envoi où il redémarre en mode Mesh (quittant le mode OTA), se connecte au réseau Mesh public, puis envoie la trame de mise à jour (vue précédemment). Une fois la trame envoyée, tous les appareils redémarrent dans un même réseau Mesh privé.

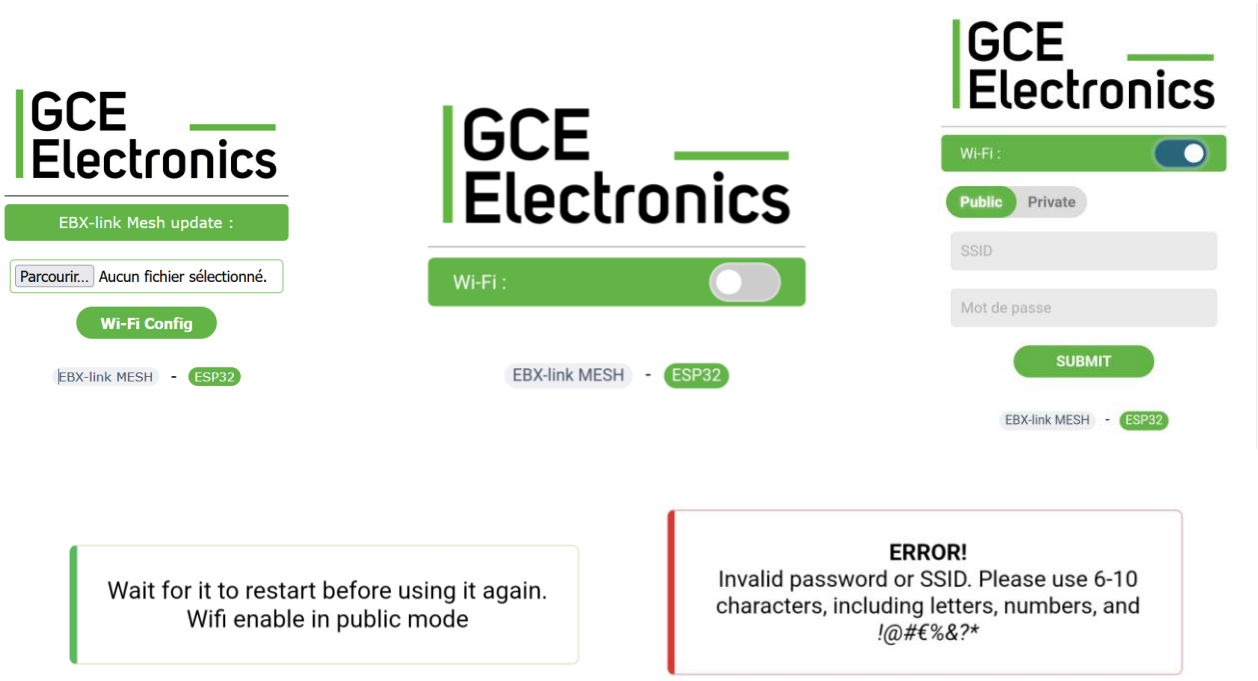


Figure 57 : Interface WEB embarquée de l'ESP Mesh

8 - Les tests et l'optimisation logicielle

Vérifier

Des tests ont été effectués tout au long du développement, ils font partie intégrante du processus de développement : après avoir codé une partie on vérifie d'abord qu'elle fonctionne avant de passer à la suivante.

De plus, Jérémy avait emmené un prototype chez lui. En effet, une fois les fonctions principales de l'EBX Link (Passerelle entre le réseau EBX et BLE), il a pris un prototype chez lui.
Durant plus d'un mois, et encore à ce jour, aucune des trames des capteurs de température Bluetooth n'a été perdue ou corrompue avant d'arriver à l'IPX 800. Cela signifie que les 4320 (24 × 6 × 30) trames ont bien été transmises, permettant ainsi de gérer automatiquement et efficacement l'asservissement de la température des radiateurs.

8.1 Utilisation conventionnelle

Dans le cadre d'une utilisation conventionnelle de la carte, j'ai créé un document Excel expliquant les étapes, le placement des extensions/modules et des protocoles de tests.

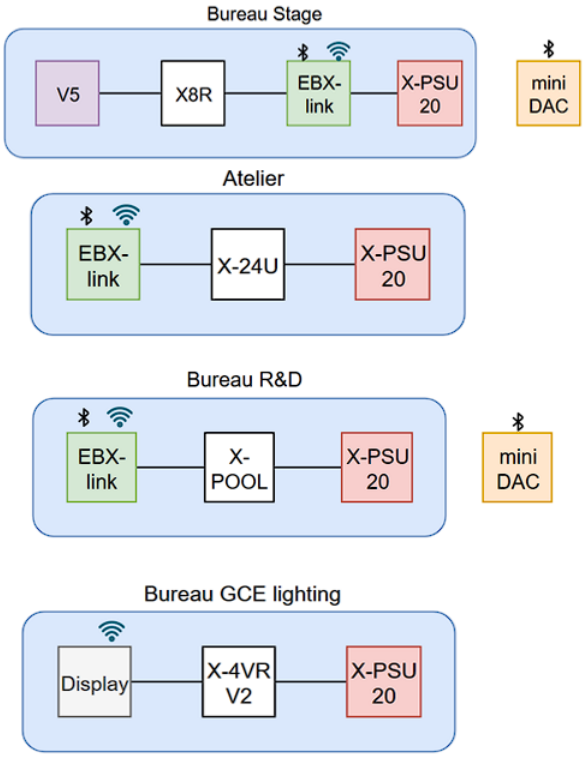


Figure 58 : Schéma de l'environnement de tests

Ces tests ont pour but de vérifier si la carte fonctionne dans la majorité des cas d'utilisation principale, pour cela j'ai branché plusieurs extensions, et relié plusieurs réseaux vers sans fils. Afin de vérifier si tout fonctionnait convenablement j'ai ajouté tous les appareils sur l'interface de l'IPX et vérifié si ceux-ci répondaient aux demandes de l'IPX 800 malgré le réseau Mesh. Dans l'ensemble, la connexion entre les appareils du réseau était fonctionnelle.

Tests spécifiques de la carte (cas extrêmes) :

○8.2 Test de Robustesse

Dans cette partie de mon stage, j'ai réalisé plusieurs tests spécifiques, notamment un test de robustesse du réseau. Pour cela, j'ai ajouté un blinker (processus qui active et désactive une fonctionnalité à intervalles réguliers) sur l'IPX 800 V5, relié à une X-8R via le réseau Mesh. L'un des relais de la X-8R s'allumait et s'éteignait toutes les 500 ms. À chaque cycle, on peut voir la LED verte s'activer en synchronisation. Après optimisation, le système fonctionnait avec une latence stable et sans variations significatives.



Figure 60 : Extension X-8R lorsqu'elle reçoit une trame



Figure 59 : Extension X-8R lorsqu'elle ne reçoit pas de trame

○8.3 Test de couverture

Ensuite, pour réaliser un test de couverture, cela n'a pas été très compliqué : il a suffi d'éloigner le plus possible deux EBX-Link. Une passerelle a été placée dans mon bureau et l'autre au fond du bâtiment, à une distance de plus de 20 mètres. Les deux cartes ont bien réussi à se connecter entre elles et j'ai même pu actionner une X-8R reliée à l'EBX-Link."

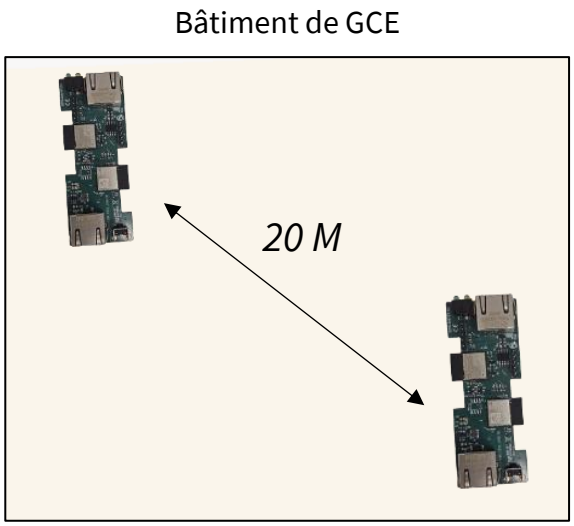


Figure 61 : Schéma du test de couverture

8.4 Phase d'optimisation finale

Concevoir Maintenir

En toute fin de stage, après une vérification du code par Jérémy, une phase d'optimisation a permis d'améliorer les performances du réseau Mesh notamment et de pérenniser le code afin qu'il soit bien compris par Jérémy pour ses futures mises à jour et facilement transposable vers d'autres extensions.
En effet, avant de finir le stage, j'ai repris la partie configuration mesh via une interface web de l'ESP Mesh afin de l'adapter à tous les appareils Mesh de la gamme, pour mettre à jour depuis n'importe quelle extension les informations du réseau Mesh et faciliter à l'utilisateur la création d'un réseau privé.

Conclusion

Bilan du travail réalisé pendant ces 10 semaines

Ce stage chez GCE Electronics m'a offert une immersion complète dans le développement d'un produit électronique, de la conception aux tests finaux. Mon travail sur l'extension EBX-Link m'a permis d'aborder la majorité des étapes d'un projet : conception matérielle et logicielle, production des prototypes et tests.

La partie matérielle a nécessité plusieurs semaines de réflexion sur le routage du PCB et la gestion de la CEM. Côté logiciel, des solutions innovantes ont été développées permettant une communication fluide entre les réseaux EBX Mesh et Bluetooth, tout en mettant en place des uns moyens de mise à jour des ESP.

Les tests ont confirmé la fiabilité du système, notamment en conditions réelles, où aucune trame n'a été perdue sur plus d'un mois de fonctionnement continu. Ces résultats valident les choix techniques effectués et démontrent la capacité de l'EBX-Link en tant que passerelle performent entre différents protocoles de communication. Cela valide et ouvre la voie vers la commande des PCB pour une production en série, ainsi qu'une commercialisation prochaine, ce qui représente un réel accomplissement.

Bilan personnel et perspectives professionnelle

D'un point de vue personnel, ce stage a été une expérience formatrice et professionnalisante. J'ai pu appliquer et approfondir mes connaissances apprises en GEII, tout en découvrant les exigences d'un environnement professionnel.

Travailler au sein d'une petite équipe de passionné m'a permis de développer mon autonomie, ma capacité d'analyse et mon adaptabilité face aux défis techniques.

Cette expérience renforce mon intérêt pour l'électronique connectée et m'a donné envie d'approfondir mes compétences dans ce domaine. À l'avenir, je souhaite continuer à explorer des solutions innovantes en intégrant des technologies de communication avancées et en optimisant les performances des systèmes embarqués.

De plus, cette expérience dans une entreprise m'a permis d'en apprendre beaucoup dans le domaine de l'hardware et l'électronique que j'affectionne tout particulièrement, je souhaite continuer dans cette voie en postulant en alternance dans une école d'ingénieur en électronique.

En conclusion, ce stage a été une étape importante dans mon parcours, me permettant de découvrir le monde professionnel dans mon domaine de prédilection et d'affiner mon projet académique et professionnel.

Annexes

Annexe 1 :

Exemple de la création d'une trame EBX (X-THL) ainsi que le pseudo-code utilisé pour sa création avec l'ESP BLE. Il est nécessaire d'effectuer des conversions afin que la température et la batterie soit correctement reçu et affiché par l'IPX 800 V5. Ce pseudo-code détaille la logique mise en place pour collecter et envoyer les données de capteur Bluetooth et sous forme d'une trame EBX.

```
Fonction UART_Send(buff, size):
Commencer
    Envoyer le début de la trame : START_EBX
    Envoyer le type de trame : XTHL
    Envoyer la longueur de la trame : LENGTH

    Récupérer les 3 dernières parties de l'adresse MAC
    Envoyer le caractère converti via UART (Serial1)

    // Lire les valeurs de température
    TempSIGN = buff[17]  // Signe de la température
    tempHB = buff[18]    // Bit de poids fort de la température
    tempLB = buff[19]    // Bit de poids faible la température
    // Puis les convertir
    Combiner les partie fort et faible pour obtenir la température
    Convertir la température dans le format comptaible IPX (via _sensors_ConvertTemperatureForIPX)
    Envoyer la température convertie via Serial1

    // Lire et convertir les valeurs de batterie
    batteryPercent = buff[20]
    Convertir la batterie dans le format IPX (via _sensors_ConvertBatteryForIPX)
    Envoyer la température convertie via Serial1


    // Bourrage (0) pour les autres capteurs (Luminosité)
    Envoyer des valeurs de remplissage pour Luminosité
    Envoyer la fin du trame : STOP
Terminer

```

Figure A1 : Pseudo-code de la fonction d'envoi de tram EBX (ESP BLE)

Annexe 2 :

Pour les calculs afin de convertir la valeur hexadécimale de la température en binaire pour l'IPX 800 V5, Jérémy m'a donné la fonction de transfert du capteur de température :

$$(\frac{V_{Hexa} \times 175,72}{65535}) - 46,85 = V_{température\ conbinée}$$

On peut alors effectuer les calculs pour un exemple de conversion :

Trame EBX : `2A 30 1A 00 00 05 67 C9 D9 15 00 00 0D`

$$\left(\frac{0x67C9 \times 175,72}{65535}\right) - 46,85 = 23,39°C$$

Enfin, pour la batterie restante la fonction de transfert est similaire :

$$\frac{V_{Hexa} \times 175,72}{65535}) - 46,85 = = V_{Batterie}$$

Annexe 3 :

Voici le premier arrangement de composants que j'ai effectué sur la carte EBX-Link, on peut voir qu'on retrouve le placement initial des composants principaux. Sur la figure A3, qui présente la maquette du placement des composants sur la carte, on constate que cette configuration est assez similaire à celle du PCB final (figure 15)."

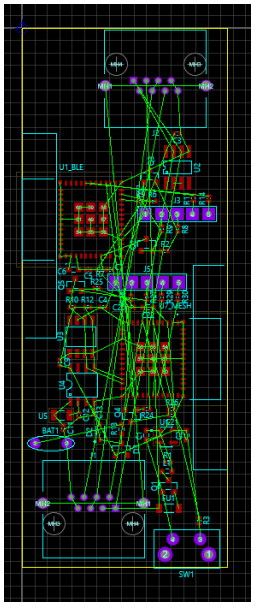


Figure A2 : Premier configuration de composants sur Proteus de la carte EBX-Link

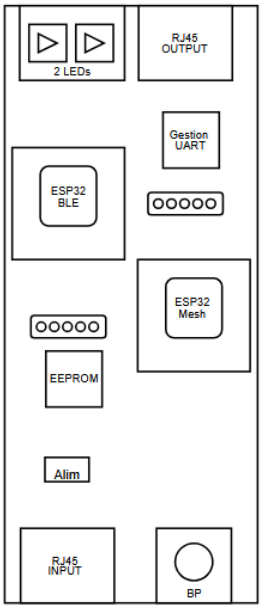


Figure A3 : Maquette sur draw.io du PCB de l'EBX-LINK

Annexe 4 :

Base 64 : Base64 est l'algorithme de conversion binaire-vers-texte le plus populaire, utilisé pour transformer des données en texte brut afin d'éviter toute corruption lors de leur transmission entre différents supports de stockage. De plus, il est souvent employé pour intégrer des données binaires dans des documents textuels tels que HTML, CSS, JavaScript ou XML.

Annexe 5 :

Pour les plus curieux, je vous invite à visiter le site web de GCE Electronics afin d'en apprendre davantage sur le rôle de chaque extension :

- IPX 800 V5 : Un automate Websever
- X-PSU20 : Alimentation Powered EBX
- X8R Connect : Relais connectées
- X-Pool Connect : Gestion de piscine en toute simplicité
- X-4VR V2 : Pilotage de volets roulants
- X-Display 2 : Thermostat connecté
- X-24U : Entrées Optocouplées

Annexe 6 :

Description des différentes machines présentes chez GCE Electronics :



Cette première machine a pour rôle d'effectuer l'étape de sérigraphie des cartes électroniques de façon automatique et avec une précision de 100 %. Une fois la carte sérigraphiée, elle est directement envoyée à la machine suivante.



Une fois la pâte à braser étalée sur les cartes, la deuxième machine permet de placer les composants à une vitesse fulgurante. C'est vraiment très impressionnant, surtout comparé au temps que cela prend lorsque c'est fait manuellement.



Une fois les composants placés, les techniciens placent les cartes dans le four à refusion. Celui-ci est particulier, car il fonctionne sous oxygène, afin de limiter l'oxydation de l'étain durant la phase de fusion. Il valide également les recommandations nécessaires pour la fabrication de l'électronique spatial.



Enfin, la plus récente est une machine très satisfaisante à fontaine d'étain. Elle permet d'effectuer la soudure des composants traversants de manière très rapide et parfaitement propre. (Je n'ai malheureusement pas la photo de celle-ci)

Bibliographie

Ensemble des documents utilisés pour rédiger ce rapport :

- Scribbr – Couverture de rapport de stage, paru le 7 décembre 2020. Lien : https://www.scribbr.fr/rapport-de-stage/couverture-de-rapport-de-stage
- Base64.guru – Home, consulté le 20 février 2025. Disponible sur : https://base64.gur
- RS Online – Accessoires pour outils de développement, consulté le 8 Janvier 2025. Lien : https://fr.rs-online.com/web/p/accessoires-pour-outils-de-developpement/0429307
- Espressif Systems – Bootloader des ESP32, consulté le 9 Janvier 2025. Lien : https://docs.espressif.com/projects/esptool/en/latest/esp32/advanced-topics/boot-mode-selection.html#manual-bootloader
- Espressif Systems – ESP32-C3-MINI-1 Datasheet, consulté le 9 Janvier 2025. Lien : https://www.espressif.com/sites/default/files/documentation/esp32-c3-mini-1_datasheet_en.pdf
- Espressif Systems – ESP-Prog Guide, consulté le 9 Janvier 2025. Lien : https://docs.espressif.com/projects/esp-iot-solution/en/latest/hw-reference/ESP-Prog_guide.html?utm_source=chatgpt.com
- GCE Electronics – Site officiel, consulté tout au long du stage. Lien : https://www.gce-electronics.com/fr

Résumé

Durant mon stage chez GCE Electronics, j'ai conçu et développé l'extension EBX-LINK, une carte électronique facilitant la communication entre les équipements connectés via Bluetooth Mesh et RJ45. Ce projet m'a permis de participer à toutes les étapes du développement, de la conception matérielle (PCB, alimentation, routage) au développement logiciel (programmation des ESP32, mise en place des protocoles de communication). Mais aussi, à la création de prototype, tests sur un réseau réaliste dans un but de production en série de la carte.

Ce stage a été une expérience enrichissante, me permettant d'approfondir mes compétences en électronique embarquée, conception hardware et communication sans fil, tout en évoluant dans un cadre professionnel familiale et dynamique.

Mots-clés :

Électronique embarquée - Conception de PCB - Communication sans fil – Bluetooth - EBX Mesh - EBX Connect - ESP32 - Développement embarqué - Automate connecté - Optimisation CEM